C++ Standard Library Issues to be moved in Kona

Doc. no. P2703R0

Date: 2022-11-07

Audience: WG21
Reply to: Jonathan Wakely <lwgchair@gmail.com>

Ready Issues

3118. `fpos` equality comparison unspecified

Section: 31.5.3.2 [fpos.operations] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2018-06-04 **Last modified:** 2022-11-02

Priority: 4

View all other issues in [fpos.operations].

Discussion:

The `fpos` requirements do not give any idea what is compared by `operator==` (even after Daniel's P0759R1 paper). I'd like something to make it clear that `return true;` is not a valid implementation of `operator==(const fpos<T>&, const fpos<T>&)`. Maybe in the `P(o)` row state that "`p == P(o)`" and "`p != P(o + 1)`", i.e. two `fpos` objects constructed from the same `streamoff` values are equal, and two `fpos` objects constructed from two different `streamoff` values are not equal.

[2018-06-23 after reflector discussion]

Priority set to 4

[2022-05-01; Daniel comments and provides wording]

The proposed wording does intentionally not use a form involving addition or subtraction to prevent the need for extra wording that ensures that this computed value is well-defined. According to 31.2.2 [stream.types], `streamoff` is a signed basic integral type, so we know what equality means for such values.

Previous resolution [SUPERSEDED]:

This wording is relative to N4910.

-

Modify in 31.5.3.2 [fpos.operations] as indicated:

> [*Drafting note:* The return type specification of `operator==` should be resolved in sync with D2167R2; see also LWG 2114.]
>
> (1.1) — […]
>
> […]
>
> (1.5) — `o` and `o2` refer~~s~~ to ~~a~~ value~~s~~ of type `streamoff` or `const streamoff`.

Table 119: Position type requirements [tab:fpos.operations]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
…			
`P p(o);` `P p = o;`			*Effects:* Value-initializes the state object. *Postconditions:* `p == P(o)` is `true`.
…			
`O(p)`	`streamoff`	converts to `offset`	`P(O(p)) == p`
`p == q`	convertible to `bool`		*Remarks:* For any two values `o` and `o2`, if `p` is obtained from `o` converted to `P` or from a copy of such `P` value and if `q` is obtained from `o2`

			converted to `P` or from a copy of such `P` value, then `bool(p == q)` is `true` only if `o == o2` is `true`.
`p != q`	convertible to `bool`	`!(p == q)`	
…			

[2022-11-02; Daniel comments and improves wording]

LWG discussion of P2167R2 has shown preference to require that the equality operations of `fpos` should be specified to have type `bool` instead of being specified as "convertible to `bool`". This has been reflected in the most recent paper revision P2167R3. The below wording changes follow that direction to reduce the wording mismatch to a minimum.

[2022-11-02; LWG telecon]

Moved to Ready. For: 6, Against: 0, Neutral: 0

Proposed resolution:

This wording is relative to N4917.

-

Modify in 31.5.3.2 [fpos.operations] as indicated:

(1.1) — […]

[…]

(1.5) — `o` and `o2` refer~~s~~ to ~~a~~ value~~s~~ of type `streamoff` or `const streamoff`.

Table 124: Position type requirements [tab:fpos.operations]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
…			
`P p(o);` `P p = o;`			*Effects:* Value-initializes the state object. *Postconditions:* `p == P(o)` is `true`.
…			
`O(p)`	`streamoff`	converts to `offset`	`P(O(p)) == p`
`p == q`	`bool`		*Remarks:* For any two values `o` and `o2`, if `p` is obtained from `o` converted to `P` or from a copy of such `P` value and if `q` is obtained from `o2` converted to `P` or from a copy of such `P` value, then `p == q` is `true` only if `o == o2` is `true`.
`p != q`	~~convertible to~~ `bool`	`!(p == q)`	
…			

3594. `inout_ptr` — inconsistent `release()` in destructor

Section: 20.3.4.3 [inout.ptr.t] **Status:** Ready **Submitter:** JeanHeyd Meneide **Opened:** 2021-09-16 **Last modified:** 2022-08-24

Priority: 1

Discussion:

More succinctly, the model for `std::out_ptr_t` and `std::inout_ptr_t` both have a conditional release in their destructors as part of their specification (20.3.4.1 [out.ptr.t] p8) (Option #2 below). But, if the wording is followed, they have an unconditional release in their constructor (Option #1 below). This is not exactly correct and can cause issues with double-free in `inout_ptr` in particular.

Consider a function `MyFunc` that sets `rawPtr` to `nullptr` when freeing an old value and deciding not to produce a new value, as shown below:

```
// Option #1:
auto uptr = std::make_unique<BYTE[]>(25);
auto rawPtr = uptr.get();
uptr.release(); // UNCONDITIONAL
MyFunc(&rawPtr);
If (rawPtr)
{
  uptr.reset(rawPtr);
}

// Option #2:
auto uptr = std::make_unique<BYTE[]>(25);
auto rawPtr = uptr.get();
MyFunc(&rawPtr);
If (rawPtr)
```

```
{
  uptr.release(); // CONDITIONAL
  uptr.reset(rawPtr);
}
```

This is no problem if the implementation selected Option #1 (release in the constructor), but results in double-free if the implementation selected option #2 (release in the destructor).

As the paper author and after conferring with others, the intent was that the behavior was identical and whether a choice between the constructor or destructor is made. The reset should be unconditional, at least for `inout_ptr_t`. Suggested change for the `~inout_ptr_t` destructor text is to remove the "`if (p) { ... }`" wrapper from around the code in 20.3.4.3 [inout.ptr.t] p11.

[2021-09-24; Reflector poll]

Set priority to 1 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to N4892.

1. Modify 20.3.4.3 [inout.ptr.t] as indicated:

```
~inout_ptr_t();
```

-9- Let `SP` be *`POINTER_OF_OR`*`(Smart, Pointer)` (20.2.1 [memory.general]).

-10- Let *`release-statement`* be `s.release();` if an implementation does not call `s.release()` in the constructor. Otherwise, it is empty.

-11- *Effects:* Equivalent to:

(11.1) —

```
~~if (p) {~~
  apply([&](auto&&... args) {
  s = Smart( static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));
~~}~~
```

if `is_pointer_v<Smart>` is `true`;

(11.2) — otherwise,

```
~~if (p) {~~
  apply([&](auto&&... args) {
  release-statement;
  s.reset(static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));
~~}~~
```

if the expression `s.reset(static_cast<SP>(p), std::forward<Args>(args)...)` is well-formed;

(11.3) — otherwise,

```
~~if (p) {~~
  apply([&](auto&&... args) {
  release-statement;
  s = Smart(static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));
~~}~~
```

if `is_constructible_v<Smart, SP, Args...>` is `true`;

(11.4) — otherwise, the program is ill-formed.

[2021-10-28; JeanHeyd Meneide provides improved wording]

[2022-08-24 Approved unanimously in LWG telecon.]

Proposed resolution:

This wording is relative to N4901.

1. Modify 20.3.4.3 [inout.ptr.t] as indicated:

```
~inout_ptr_t();
```

-9- Let `SP` be *`POINTER_OF_OR`*`(Smart, Pointer)` (20.2.1 [memory.general]).

-10- Let *`release-statement`* be `s.release();` if an implementation does not call `s.release()` in the constructor. Otherwise, it is empty.

-11- *Effects:* Equivalent to:

(11.1) —

```
if (p) {
  apply([&](auto&&... args) {
  s = Smart( static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));
}
```

if `is_pointer_v<Smart>` is `true`;

(11.2) — otherwise,

```
release-statement;
if (p) {
  apply([&](auto&&... args) {
  release statement;
  s.reset(static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));
}
```

if the expression `s.reset(static_cast<SP>(p), std::forward<Args>(args)...)` is well-formed;

(11.3) — otherwise,

```
release-statement;
if (p) {
  apply([&](auto&&... args) {
  release statement;
  s = Smart(static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));
}
```

if `is_constructible_v<Smart, SP, Args...>` is `true`;

(11.4) — otherwise, the program is ill-formed.

3636. `formatter<T>::format` should be `const`-qualified

Section: 22.14.6.1 [formatter.requirements] **Status:** Ready **Submitter:** Arthur O'Dwyer **Opened:** 2021-11-11 **Last modified:** 2022-08-24

Priority: 1

View other active issues in [formatter.requirements].

View all other issues in [formatter.requirements].

Discussion:

In libc++ review, we've noticed that we don't understand the implications of 22.14.6.1 [formatter.requirements] bullet 3.1 and Table [tab:formatter.basic]: (emphasize mine):

> (3.1) — `f` is a **value** of type `F`,
>
> […]
>
> Table 70: *BasicFormatter* requirements [tab:formatter.basic]
>
> […]
>
> `f.parse(pc)` [*must compile*] […]
>
> `f.format(u, fc)` [*must compile*] […]

According to Victor Zverovich, his intent was that `f.parse(pc)` should modify the state of `f`, but `f.format(u, fc)` should merely read `f`'s state to support format string compilation where formatter objects are immutable and therefore the `format` function must be `const`-qualified.

That is, a typical formatter should look something like this (modulo errors introduced by me):

```
struct WidgetFormatter {
  auto parse(std::format_parse_context&) -> std::format_parse_context::iterator;
  auto format(const Widget&, std::format_context&) const -> std::format_context::iterator;
};
```

However, this is not reflected in the wording, which treats `parse` and `format` symmetrically. Also, there is at least one example that shows a non-const `format` method:

```
template<> struct std::formatter<color> : std::formatter<const char*> {
  auto format(color c, format_context& ctx) {
    return formatter<const char*>::format(color_names[c], ctx);
  }
};
```

Victor writes:

> Maybe we should […] open an LWG issue clarifying that all standard formatters have a `const` format function.

I'd like to be even more assertive: Let's open an LWG issue clarifying that all formatters must have a `const` format function!

[2022-01-30; Reflector poll]

Set priority to 1 after reflector poll.

[2022-08-24 Approved unanimously in LWG telecon.]

Proposed resolution:

This wording is relative to N4901.

1. Modify 22.14.6.1 [formatter.requirements] as indicated:

> [*Drafting note:* It might also be reasonable to do a drive-by clarification that when the Table 70 says "Stores the parsed format specifiers in `*this`," what it actually means is "Stores the parsed format specifiers in `g`." (But I don't think anyone's seriously confused by that wording.)

-3- Given character type `charT`, output iterator type `Out`, and formatting argument type `T`, in Table 70 and Table 71:

(3.1) — `f` is a value of type (possibly `const`) `F`,

(3.?) — `g` is an lvalue of type `F`,

(3.2) — `u` is an lvalue of type `T`,

(3.3) — `t` is a value of a type convertible to (possibly `const`) `T`,

[…]

[…]

Table 70: *Formatter* requirements [tab:formatter]

Expression	Return type	Requirement
`~~f~~g.parse(pc)`	`PC::iterator`	[…] Stores the parsed format specifiers in `*this` and returns an iterator past the end of the parsed range.
…		

2. Modify 22.14.6.3 [format.formatter.spec] as indicated:

-6- An enabled specialization `formatter<T, charT>` meets the *BasicFormatter* requirements (22.14.6.1 [formatter.requirements]).

[*Example 1*:

```
#include <format>

enum color { red, green, blue };
const char* color_names[] = { "red", "green", "blue" };

template<> struct std::formatter<color> : std::formatter<const char*> {
  auto format(color c, format_context& ctx) const {
    return formatter<const char*>::format(color_names[c], ctx);
  }
};

[…]
```

— *end example*]

3. Modify 22.14.6.6 [format.context] as indicated:

```
void advance_to(iterator it);
```

-8- *Effects:* Equivalent to: `out_ = std::move(it);`

[*Example 1*:

```
struct S { int value; };

template<> struct std::formatter<S> {
  size_t width_arg_id = 0;

  // Parses a width argument id in the format { digit }.
```

```
    constexpr auto parse(format_parse_context& ctx) {
      […]
    }

    // Formats an S with width given by the argument width_arg_id.
    auto format(S s, format_context& ctx) const {
      int width = visit_format_arg([](auto value) -> int {
        if constexpr (!is_integral_v<decltype(value)>)
          throw format_error("width is not integral");
        else if (value < 0 || value > numeric_limits<int>::max())
          throw format_error("invalid width");
        else
          return value;
        }, ctx.arg(width_arg_id));
      return format_to(ctx.out(), "{0:x<{1}}", s.value, width);
    }
  };

  […]
```

— *end example*]

4. Modify 29.12 [time.format] as indicated:

```
template<class Duration, class charT>
struct formatter<chrono::local-time-format-t<Duration>, charT>;
```

-15- Let `f` be […]

-16- *Remarks:* […]

```
template<class Duration, class TimeZonePtr, class charT>
struct formatter<chrono::zoned_time<Duration, TimeZonePtr>, charT>
  : formatter<chrono::local-time-format-t<Duration>, charT> {
  template<class FormatContext>
    typename FormatContext::iterator
      format(const chrono::zoned_time<Duration, TimeZonePtr>& tp, FormatContext& ctx) const;
};

template<class FormatContext>
  typename FormatContext::iterator
    format(const chrono::zoned_time<Duration, TimeZonePtr>& tp, FormatContext& ctx) const;
```

-17- *Effects:* Equivalent to:

```
sys_info info = tp.get_info();
return formatter<chrono::local-time-format-t<Duration>, charT>::
         format({tp.get_local_time(), &info.abbrev, &info.offset}, ctx);
```

3754. Class template `expected` synopsis contains declarations that do not match the detailed description

Section: 22.8.6.1 [expected.object.general] **Status:** Ready **Submitter:** S. B. Tam **Opened:** 2022-08-23 **Last modified:** 2022-09-07

Priority: Not Prioritized

Discussion:

22.8.6.1 [expected.object.general] declares the following constructors:

```
template<class G>
  constexpr expected(const unexpected<G>&);
template<class G>
  constexpr expected(unexpected<G>&&);
```

But in 22.8.6.2 [expected.object.ctor], these constructors are declared as:

```
template<class G>
  constexpr explicit(!is_convertible_v<const G&, E>) expected(const unexpected<G>& e);
template<class G>
  constexpr explicit(!is_convertible_v<G, E>) expected(unexpected<G>&& e);
```

Note that they have no explicit-specifiers in 22.8.6.1 [expected.object.general], but are conditionally explicit in 22.8.6.2 [expected.object.ctor].

I presume that 22.8.6.1 [expected.object.general] is missing a few `explicit(`*`see below`*`)`.

The same inconsistency exists in 22.8.7 [expected.void].

[2022-09-05; Jonathan Wakely provides wording]

In N4910 the `expected` synopses had `explicit(`*`see below`*`)` on the copy and move constructors. That was fixed editorially, but this other inconsistency was not noticed.

[2022-09-07; Moved to "Ready" at LWG telecon]

Proposed resolution:

This wording is relative to N4917.

1. Change 22.8.6.1 [expected.object.general] as indicated:

```
// 22.8.6.2, constructors
constexpr expected();
constexpr expected(const expected&);
constexpr expected(expected&&) noexcept(see below);
template<class U, class G>
  constexpr explicit(see below) expected(const expected<U, G>&);
template<class U, class G>
  constexpr explicit(see below) expected(expected<U, G>&&);

template<class U = T>
  constexpr explicit(see below) expected(U&& v);

template<class G>
  constexpr explicit(see below) expected(const unexpected<G>&);
template<class G>
  constexpr explicit(see below) expected(unexpected<G>&&);

template<class... Args>
  constexpr explicit expected(in_place_t, Args&&...);
```

2. Change 22.8.7.1 [expected.void.general] as indicated:

```
// 22.8.7.2, constructors
constexpr expected() noexcept;
constexpr expected(const expected&);
constexpr expected(expected&&) noexcept(see below);
template<class U, class G>
  constexpr explicit(see below) expected(const expected<U, G>&&);

template<class G>
  constexpr explicit(see below) expected(const unexpected<G>&);
template<class G>
  constexpr explicit(see below) expected(unexpected<G>&&);

constexpr explicit expected(in_place_t) noexcept;
```

Tentatively Ready Issues

3028. Container requirements tables should distinguish `const` and non-`const` variables

Section: 24.2.2.1 [container.requirements.general] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2017-10-17 **Last modified:** 2022-09-05

Priority: 3

View other active issues in [container.requirements.general].

View all other issues in [container.requirements.general].

Discussion:

[container.requirements.general] p4 says:

> In Tables 83, 84, and 85 `X` denotes a container class containing objects of type `T`, `a` and `b` denote values of type `X`, `u` denotes an identifier, `r` denotes a non-`const` value of type `X`, and `rv` denotes a non-`const` rvalue of type `X`.

This doesn't say anything about whether `a` and `b` are allowed to be `const`, or must be non-`const`. In fact Table 83 uses them inconsistently, e.g. the rows for "`a = rv`" and "`a.swap(b)`" most certainly require them to be non-`const`, but all other uses are valid for either `const` or non-`const` `X`.

[2017-11 Albuquerque Wednesday night issues processing]

Priority set to 3; Jonathan to provide updated wording.

Wording needs adjustment - could use "possibly const values of type X"

Will distinguish between lvalue/rvalue

Previous resolution [SUPERSEDED]:

> This wording is relative to N4687.

1. Change 24.2.2.1 [container.requirements.general] p4 as indicated:

> -4- In Tables 83, 84, and 85 `X` denotes a container class containing objects of type `T`, `a` and `b` denote values of type `X`, `u` denotes an identifier, `r` and `s` denote~~s `a`~~ non-`const` value~~s~~ of type `X`, and `rv` denotes a non-`const` rvalue of type `X`.

2. Change 24.2.2.1 [container.requirements.general], Table 83 "Container requirements", as indicated:

Table 83 — Container requirements

Expression	Return type	Operational semantics	Assertion/note pre/post-condition	Complexity
[…]				
~~`a`~~`r = rv`	`X&`	All existing elements of ~~`a`~~`r` are either move assigned to or destroyed	~~`a`~~`r` shall be equal to the value that `rv` had before this assignment	linear
[…]				
~~`a`~~`r.swap(~~`b`~~s)`	`void`		exchanges the contents of ~~`a`~~`r` and ~~`b`~~`s`	(Note A)
[…]				
`swap(`~~`a`~~`r, ~~`b`~~s)`	`void`	~~`a`~~`r.swap(~~`b`~~s)`		(Note A)

[2020-05-03; Daniel provides alternative wording]

Previous resolution [SUPERSEDED]:

This wording is relative to N4861.

1. Change 24.2.2.1 [container.requirements.general] as indicated:

> [*Drafting note:*
>
> - The following presentation also transforms the current list into a bullet list as we already have in 24.2.8 [unord.req] p11
> - It has been decided to replace the symbol `r` by `s`, because it is easy to confuse with `rv` but means an lvalue instead, and the other container tables use it rarely and for something completely different (iterator value)
> - A separate symbol `v` is introduced to unambigiously distinguish the counterpart of a non-`const` rvalue (See 16.4.4.2 [utility.arg.requirements])
> - Two separate symbols `b` and `c` represent now "(possibly `const`) values, while the existing symbol `a` represents an unspecified value, whose meaning becomes defined when context is provided, e.g. for overloads like `begin()` and `end`
>
> -4- In Tables 73, 74, and 75**:**
>
> (4.1) — `X` denotes a container class containing objects of type `T`,
>
> (4.2) — a ~~and `b`~~ denote~~s~~ a value~~s~~ of type `X`,
>
> (4.2) — `b` and `c` denote (possibly `const`) values of type `X`,
>
> (4.3) — `i` and `j` denote values of type (possibly `const`) `X::iterator`,
>
> (4.4) — `u` denotes an identifier,
>
> (?.?) — `v` denotes an lvalue of type (possibly `const`) `X` or an rvalue of type `const X`,
>
> (4.5) — ~~`r`~~`s` and `t` denote~~s `a`~~ non-`const` ~~value~~lvalues of type `X`, and
>
> (4.6) — `rv` denotes a non-`const` rvalue of type `X`.

2. Change 24.2.2.1 [container.requirements.general], Table 73 "Container requirements" [tab:container.req], as indicated:

> [*Drafting note:* The following presentation also moves the copy-assignment expression just before the move-assignment expression]

Table 73: — Container requirements [tab:container.req]

Expression	Return type	Operational semantics	Assertion/note pre/post-condition	Complexity

[…]				
`X(~~a~~v)`			*Preconditions:* `T` is *Cpp17CopyInsertable* into `X` (see below). *Postconditions:* `~~a~~v == X(~~a~~v)`.	linear
`X u(~~a~~v);` `X u = ~~a~~v;`			*Preconditions:* `T` is *Cpp17CopyInsertable* into `X` (see below). *Postconditions:* `u == ~~a~~v`.	linear
`X u(rv);` `X u = rv;`			*Postconditions:* `u` is equal to the value that `rv` had before this construction	(Note B)
`t = v`	`X&`		*Postconditions:* `t == v`.	linear
`~~a~~t = rv`	`X&`	All existing elements of `~~a~~t` are either move assigned to or destroyed	`~~a~~t` shall be equal to the value that `rv` had before this assignment	linear
[…]				
`~~a~~c == b`	convertible to `bool`	`==` is an equivalence relation. `equal(~~a~~c.begin(), ~~a~~c.end(), b.begin(), b.end())`	*Preconditions:* `T` meets the *Cpp17EqualityComparable* requirements	Constant if `~~a~~c.size() != b.size()`, linear otherwise
`~~a~~c != b`	convertible to `bool`	Equivalent to `!(~~a~~c == b)`		linear
`~~a~~t.swap(~~b~~s)`	`void`		exchanges the contents of `~~a~~t` and `~~b~~s`	(Note A)
`swap(~~a~~t, ~~b~~s)`	`void`	`~~a~~t.swap(~~b~~s)`		(Note A)
`~~r = a~~`	`~~X&~~`		`~~*Postconditions:* r == a.~~`	`~~linear~~`
`~~a~~c.size()`	`size_type`	`distance(~~a~~c.begin(), ~~a~~c.end())`		constant
`~~a~~c.max_size()`	`size_type`	`distance(begin(), end())` for the largest possible container		constant
`~~a~~c.empty()`	convertible to `bool`	`~~a~~c.begin() == ~~a~~c.end()`		constant

[2022-04-20; Jonathan rebases the wording on the latest draft]

[2022-09-05; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll in April 2022.

Proposed resolution:

This wording is relative to N4910.

1. Change 24.2.2.1 [container.requirements.general] as indicated:

[*Drafting note:*

- It has been decided to replace the symbol `r` by `s`, because it is easy to confuse with `rv` but means an lvalue instead, and the other container tables use it rarely and for something completely different (iterator value)
- A separate symbol `v` is introduced to unambigiously distinguish the counterpart of a non-`const` rvalue (See 16.4.4.2 [utility.arg.requirements])
- Two separate symbols `b` and `c` represent now "(possibly `const`) values, while the existing symbol `a` represents an unspecified value, whose meaning becomes defined when context is provided, e.g. for overloads like `begin()` and `end`

-1- In subclause 24.2.2 [container.gen.reqmts],

(1.1) — `X` denotes a container class containing objects of type `T`,

(1.2) — a ~~and `b` denote values~~ denotes a value of type `X`,

(?.?) — `b` and `c` denote values of type (possibly `const`) `X`,

(1.3) — `i` and `j` denote values of type (possibly `const`) `X::iterator`,

(1.4) — `u` denotes an identifier,

(?.?) — `v` denotes an lvalue of type (possibly `const`) `X` or an rvalue of type `const X`,

(1.5) — ~~`r` denotes a~~s and `t` denote non-`const` ~~value~~lvalues of type `X`, and

(1.6) — `rv` denotes a non-`const` rvalue of type `X`.

2. Change 24.2.2.2 [container.reqmts] as indicated:

> [*Drafting note:* The following presentation also moves the copy-assignment expression just before the move-assignment expression]

```
X u(av);
X u = av;
```

-12- *Preconditions:* `T` is *Cpp17CopyInsertable* into `X` (see below).

-13- *Postconditions:* `u == av`.

-14- *Complexity:* Linear.

```
X u(rv);
X u = rv;
```

-15- *Postconditions:* `u` is equal to the value that `rv` had before this construction.

-14- *Complexity:* Linear for `array` and constant for all other standard containers.

```
t = v
```

-?- *Result:* `X&`.

-?- *Postconditions:* `t == v`.

-?- *Complexity:* Linear.

```
at = rv
```

-17- *Result:* `X&`.

-18- *Effects:* All existing elements of `at` are either move assigned to or destroyed.

-19- *Postconditions:* `at` shall be equal to the value that `rv` had before this assignment.

-20- *Complexity:* Linear.

[…]

```
ab.begin()
```

-24- *Result:* `iterator`; `const_iterator` for constant `ab`.

-25- *Returns:* An iterator referring to the first element in the container.

-26- *Complexity:* Constant.

```
ab.end()
```

-27- *Result:* `iterator`; `const_iterator` for constant `ab`.

-28- *Returns:* An iterator which is the past-the-end value for the container.

-29- *Complexity:* Constant.

```
ab.cbegin()
```

-30- *Result:* `const_iterator`.

-31- *Returns:* `const_cast<X const&>(ab).begin()`

-32- *Complexity:* Constant.

```
ab.cend()
```

-33- *Result:* `const_iterator`.

-34- *Returns:* `const_cast<X const&>(ab).end()`

-35- *Complexity:* Constant.

[…]

`~~a~~c == b`

-39- *Preconditions:* `T` meets the *Cpp17EqualityComparable* requirements.

-40- *Result:* Convertible to `bool`.

-41- *Returns:* `equal(~~a~~c.begin(), ~~a~~c.end(), b.begin(), b.end())`.

[*Note 1:* The algorithm `equal` is defined in 27.6.13 [alg.equal]. — *end note*]

-42- *Complexity:* Constant if `~~a~~c.size() != b.size()`, linear otherwise.

-43- *Remarks:* `==` is an equivalence relation.

`~~a~~c != b`

-44- *Effects:* Equivalent to `!(~~a~~c == b)`.

`~~a~~t.swap(~~b~~s)`

-45- *Result:* `void`.

-46- *Effects:* Exchanges the contents of `~~a~~t` and `~~b~~s`.

-47- *Complexity:* Linear for `array` and constant for all other standard containers.

`swap(~~a~~t, ~~b~~s)`

-48- *Effects:* Equivalent to `~~a~~t.swap(~~b~~s)`

~~`r = a`~~

~~-49- *Result:* `x&`.~~

~~-50- *Postconditions:* `r == a`.~~

~~-51- *Complexity:* Linear.~~

`~~a~~c.size()`

-52- *Result:* `size_type`.

-53- *Returns:* `distance(~~a~~c.begin(), ~~a~~c.end())`, i.e. the number of elements in the container.

-54- *Complexity:* Constant.

-55- *Remarks:* The number of elements is defined by the rules of constructors, inserts, and erases.

`~~a~~c.max_size()`

-56- *Result:* `size_type`.

-57- *Returns:* `distance(begin(), end())` for the largest possible container.

-58- *Complexity:* Constant.

`~~a~~c.empty()`

-59- *Result:* Convertible to `bool`.

-60- *Returns:* `~~a~~c.begin() == ~~a~~c.end())`

-61- *Complexity:* Constant.

-62- *Remarks:* If the container is empty, then `~~a~~c.empty()` is `true`.

3177. Limit permission to specialize variable templates to program-defined types

Section: 16.4.5.2.1 [namespace.std] **Status:** Tentatively Ready **Submitter:** Johel Ernesto Guerrero Peña **Opened:** 2018-12-11 **Last modified:** 2022-09-29

Priority: 3

View other active issues in [namespace.std].

View all other issues in [namespace.std].

Discussion:

The permission denoted by [namespace.std]/3 should be limited to program-defined types.

[2018-12-21 Reflector prioritization]

Set Priority to 3

Previous resolution [SUPERSEDED]:

> This wording is relative to N4791.
>
> 1. Change 16.4.5.2.1 [namespace.std] as indicated:
>
> > -2- Unless explicitly prohibited, a program may add a template specialization for any standard library class template to namespace `std` provided that (a) the added declaration depends on at least one program-defined type and (b) the specialization meets the standard library requirements for the original template.(footnote 174)
> >
> > -3- The behavior of a C++ program is undefined if it declares an explicit or partial specialization of any standard library variable template, except where explicitly permitted by the specification of that variable template, provided that the added declaration depends on at least one program-defined type.

[2022-08-24; LWG telecon]

Each variable template that grants permission to specialize already states requirements more precisely than proposed here anyway. For example, `disable_sized_range` only allows it for cv-unqualified program-defined types. Adding less precise wording here wouldn't be an improvement. Add a note to make it clear we didn't just forget to say something here, and to remind us to state requirements for each variable template in future.

[2022-08-25; Jonathan Wakely provides improved wording]

[2022-09-28; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4910.

- Change 16.4.5.2.1 [namespace.std] as indicated:

> -2- Unless explicitly prohibited, a program may add a template specialization for any standard library class template to namespace `std` provided that (a) the added declaration depends on at least one program-defined type and (b) the specialization meets the standard library requirements for the original template.(footnote 163)
>
> -3- The behavior of a C++ program is undefined if it declares an explicit or partial specialization of any standard library variable template, except where explicitly permitted by the specification of that variable template.
>
> [*Note 1*: The requirements on an explicit or partial specialization are stated by each variable template that grants such permission. — *end note*]

3545. `std::pointer_traits` should be SFINAE-friendly

Section: 20.2.3 [pointer.traits] **Status:** Tentatively Ready **Submitter:** Glen Joseph Fernandes **Opened:** 2021-04-20 **Last modified:** 2022-10-19

Priority: 2

View other active issues in [pointer.traits].

View all other issues in [pointer.traits].

Discussion:

P1474R1 chose to use `std::to_address` (a mechanism of converting pointer-like types to raw pointers) for contiguous iterators. `std::to_address` provides an optional customization point via an optional member in `std::pointer_traits`. However all iterators are not pointers, and the primary template of `std::pointer_traits<Ptr>` requires that either `Ptr::element_type` is valid or `Ptr` is of the form `template<T, Args...>` or the `pointer_traits` specialization is ill-formed. This requires specializing `pointer_traits` for those contiguous iterator types which is inconvenient for users. P1474 should have also made `pointer_traits` SFINAE friendly.

[2021-05-10; Reflector poll]

Priority set to 2. Send to LEWG. Daniel: "there is no similar treatment for the `rebind` member template and I think it should be clarified whether `pointer_to`'s signature should exist and in which form in the offending case."

[2022-01-29; Daniel comments]

This issue has some overlap with LWG 3665 in regard to the question how we should handle the `rebind_alloc` member template of the `allocator_traits` template as specified by 20.2.9.2 [allocator.traits.types]/11. It would seem preferable to decide for the same approach in both cases.

[2022-02-22 LEWG telecon; Status changed: LEWG → Open]

No objection to unanimous consent for Jonathan's suggestion to make `pointer_traits` an empty class when there is no `element_type`. Jonathan to provide a paper.

Previous resolution [SUPERSEDED]:

This wording is relative to N4885.

1. Modify 20.2.3.2 [pointer.traits.types] as indicated:

As additional drive-by fix the improper usage of the term "instantiation" has been corrected.

```
using element_type = see below;
```

-1- *Type:* `Ptr::element_type` if the *qualified-id* `Ptr::element_type` is valid and denotes a type (13.10.3 [temp.deduct]); otherwise, `T` if `Ptr` is a class template ~~instantiation~~specialization of the form `SomePointer<T, Args>`, where `Args` is zero or more type arguments; otherwise, ~~the specialization is ill-formed~~`pointer_traits` has no member `element_type`.

[2022-09-27; Jonathan provides new wording]

Previous resolution [SUPERSEDED]:

This wording is relative to N4917.

1. Modify 20.2.3.1 [pointer.traits.general] as indicated:

-1- The class template `pointer_traits` supplies a uniform interface to certain attributes of pointer-like types.

```
namespace std {
  template<class Ptr> struct pointer_traits {
    ~~using pointer         = Ptr;~~
    ~~using element_type    = see below;~~
    ~~using difference_type = see below;~~

    ~~template<class U> using rebind = see below;~~
    ~~static pointer pointer_to(see below r);~~

    see below;
  };

  template<class T> struct pointer_traits<T*> {
    using pointer         = T*;
    using element_type    = T;
    using difference_type = ptrdiff_t;

    template<class U> using rebind = U*;
    static constexpr pointer pointer_to(see below r) noexcept;
  };
}
```

2. Modify 20.2.3.2 [pointer.traits.types] as indicated:

-?- The definitions in this subclause make use of the following exposition-only class template and concept:

```
template<class T>
struct ptr-traits-elem // exposition only
{ };

template<class T> requires requires { typename T::element_type; }
struct ptr-traits-elem<T>
{ using type = typename T::element_type; };

template<template<class...> class SomePointer, class T, class... Args>
requires (!requires { typename SomePointer<T, Args...>::element_type; })
struct ptr-traits-elem<SomePointer<T, Args...>>
{ using type = T; };

template<class Ptr>
  concept has-elem-type = // exposition only
    requires { typename ptr-traits-elem<Ptr>::type; }
```

-?- If `Ptr` satisfies *has-elem-type*, a specialization `pointer_traits<Ptr>` generated from the `pointer_traits` primary template has the members described in 20.2.3.2 [pointer.traits.types] and 20.2.3.3 [pointer.traits.functions]; otherwise, such a specialization has no members by any of the names described in those subclauses or in 20.2.3.4 [pointer.traits.optmem].

```
using pointer = Ptr;
```

```
using element_type = see below typename ptr-traits-elem<Ptr>::type;
```

-1- *Type:* `Ptr::element_type` if the *qualified-id* `Ptr::element_type` is valid and denotes a type (13.10.3 [temp.deduct]); otherwise, `T` if `Ptr` is a class template instantiation of the form `SomePointer<T, Args>`, where `Args` is zero or more type arguments; otherwise, the specialization is ill-formed.

```
using difference_type = see below;
```

-2- *Type*: `Ptr::difference_type` if the *qualified-id* `Ptr::difference_type` is valid and denotes a type (13.10.3 [temp.deduct]); otherwise, `ptrdiff_t`.

```
template<class U> using rebind = see below;
```

-3- *Alias template*: `Ptr::rebind<U>` if the *qualified-id* `Ptr::rebind<U>` is valid and denotes a type (13.10.3 [temp.deduct]); otherwise, `SomePointer<U, Args>` if `Ptr` is a class template instantiation of the form `SomePointer<T, Args>`, where `Args` is zero or more type arguments; otherwise, the instantiation of `rebind` is ill-formed.

[2022-10-11; Jonathan provides improved wording]

[2022-10-19; Reflector poll]

Set status to "Tentatively Ready" after six votes in favour in reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 20.2.3.1 [pointer.traits.general] as indicated:

-1- The class template `pointer_traits` supplies a uniform interface to certain attributes of pointer-like types.

```
namespace std {
  template<class Ptr> struct pointer_traits {
    using pointer          = Ptr;
    using element_type     = see below;
    using difference_type = see below;

    template<class U> using rebind = see below;
    static pointer pointer_to(see below r);

    see below;
  };

  template<class T> struct pointer_traits<T*> {
    using pointer         = T*;
    using element_type    = T;
    using difference_type = ptrdiff_t;

    template<class U> using rebind = U*;
    static constexpr pointer pointer_to(see below r) noexcept;
  };
}
```

2. Modify 20.2.3.2 [pointer.traits.types] as indicated:

-?- The definitions in this subclause make use of the following exposition-only class template and concept:

```
template<class T>
struct ptr-traits-elem // exposition only
{ };

template<class T> requires requires { typename T::element_type; }
struct ptr-traits-elem<T>
{ using type = typename T::element_type; };

template<template<class...> class SomePointer, class T, class... Args>
requires (!requires { typename SomePointer<T, Args...>::element_type; })
struct ptr-traits-elem<SomePointer<T, Args...>>
{ using type = T; };

template<class Ptr>
  concept has-elem-type = // exposition only
    requires { typename ptr-traits-elem<Ptr>::type; }
```

-?- If `Ptr` satisfies *`has-elem-type`*, a specialization `pointer_traits<Ptr>` generated from the `pointer_traits` primary template has the following members as well as those described in 20.2.3.3 [pointer.traits.functions]; otherwise, such a specialization has no members by any of those names.

`using pointer = see below;`

-?- *Type*: `Ptr`.

`using element_type = see below;`

-1- *Type*: `typename ptr-traits-elem<Ptr>::type`. ~~`Ptr::element_type` if the *qualified-id* `Ptr::element_type` is valid and denotes a type (13.10.3 [temp.deduct]); otherwise, `I` if `Ptr` is a class template instantiation of the form `SomePointer<T, Args>`, where `Args` is zero or more type arguments; otherwise, the specialization is ill-formed.~~

`using difference_type = see below;`

-2- *Type*: `Ptr::difference_type` if the *qualified-id* `Ptr::difference_type` is valid and denotes a type (13.10.3 [temp.deduct]); otherwise, `ptrdiff_t`.

`template<class U> using rebind = see below;`

-3- *Alias template*: `Ptr::rebind<U>` if the *qualified-id* `Ptr::rebind<U>` is valid and denotes a type (13.10.3 [temp.deduct]); otherwise, `SomePointer<U, Args>` if `Ptr` is a class template instantiation of the form `SomePointer<T, Args>`, where `Args` is zero or more type arguments; otherwise, the instantiation of `rebind` is ill-formed.

3. Modify 20.2.3.4 [pointer.traits.optmem] as indicated:

-1- Specializations of `pointer_traits` may define the member declared in this subclause to customize the behavior of the standard library. A specialization generated from the `pointer_traits` primary template has no member by this name.

`static element_type* to_address(pointer p) noexcept;`

-1- *Returns*: A pointer of type `element_type*` that references the same location as the argument `p`.

3597. Unsigned integer types don't model *`advanceable`*

Section: 26.6.4.2 [range.iota.view] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2021-09-23 **Last modified:** 2022-10-19

Priority: 3

View other active issues in [range.iota.view].

View all other issues in [range.iota.view].

Discussion:

Unsigned integer types satisfy *`advanceable`*, but don't model it, since

> every two values of an unsigned integer type are reachable from each other, and modular arithmetic is performed on unsigned integer types,

which makes the last three bullets of the semantic requirements of *`advanceable`* (26.6.4.2 [range.iota.view]/5) can't be satisfied, and some (if not all) uses of `iota_view`s of unsigned integer types ill-formed, no diagnostic required.

Some operations that are likely to expect the semantic requirements of *`advanceable`* behave incorrectly for unsigned integer types. E.g. according to 26.6.4.2 [range.iota.view]/6 and 26.6.4.2 [range.iota.view]/16, `std::ranges::iota_view<std::uint8_t, std::uint8_t>(std::uint8_t(1)).size()` is well-defined IMO, because

> `Bound()` is `std::uint8_t(0)`, which is reachable from `std::uint8_t(1)`, and not modeling *`advanceable`* shouldn't affect the validity, as both `W` and `Bound` are integer types.

However, it returns `unsigned(-1)` on common implementations (where `sizeof(int) > sizeof(std::uint8_t)`), which is wrong.

Perhaps the semantic requirements of *`advanceable`* should be adjusted, and a refined definition of reachability in 26.6.4 [range.iota] is needed to avoid reaching `a` from `b` when `a > b` (the iterator type is also affected).

[2021-10-14; Reflector poll]

Set priority to 3 after reflector poll.

[Tim Song commented:]

The advanceable part of the issue is NAD. This is no different from NaN and `totally_ordered`, see 16.3.2.3 [structure.requirements]/8.

The part about `iota_view<uint8_t, uint8_t>(1)` is simply this: when we added "When `W` and `Bound` model ..." to 26.6.4.2 [range.iota.view]/8, we forgot to add its equivalent to the single-argument constructor. We should do that.

[2022-10-12; Jonathan provides wording]

[2022-10-19; Reflector poll]

Set status to "Tentatively Ready" after seven votes in favour in reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.6.4.2 [range.iota.view] as indicated:

```
constexpr explicit iota_view(W value);
```

-6- *Preconditions*: `Bound` denotes `unreachable_sentinel_t` or `Bound()` is reachable from `value`. When `W` and `Bound` model `totally_ordered_with`, then `bool(value <= Bound())` is `true`.

-7- *Effects*: Initializes *`value_`* with `value`.

```
constexpr iota_view(type_identity_t<W> value, type_identity_t<Bound> bound);
```

-8- *Preconditions*: `Bound` denotes `unreachable_sentinel_t` or `bound` is reachable from `value`. When `W` and `Bound` model `totally_ordered_with`, then `bool(value <= bound)` is `true`.

-9- *Effects*: Initializes *`value_`* with `value` and *`bound_`* with `bound`.

3600. Making `istream_iterator` copy constructor trivial is an ABI break

Section: 25.6.2.2 [istream.iterator.cons] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2021-09-23 **Last modified:** 2022-11-07

Priority: 3

View all other issues in [istream.iterator.cons].

Discussion:

Libstdc++ never implemented this change made between C++03 and C++11 (by N2994):

24.6.1.1 [istream.iterator.cons] p3:

```
istream_iterator(const istream_iterator<T,charT,traits,Distance>& x) = default;
```

-3- *Effects:* Constructs a copy of `x`. If `T` is a literal type, then this constructor shall be a trivial copy constructor.

This breaks our ABI, as it changes the argument passing convention for the type, meaning this function segfaults if compiled with today's libstdc++ and called from one that makes the triviality change:

```
#include <iterator>
#include <istream>

int f(std::istream_iterator<int> i)
{
  return *i++;
}
```

As a result, it's likely that libstdc++ will never implement the change.

There is no reason to require this constructor to be trivial. It was required for C++0x at one point, so the type could be literal, but that is not true in the current language. We should strike the requirement, to reflect reality. MSVC and libc++ are free to continue to define it as defaulted (and so trivial when appropriate) but we should not require it from libstdc++. The cost of an ABI break is not worth the negligible benefit from making it trivial.

Previous resolution [SUPERSEDED]:

This wording is relative to N4892.

1. Modify 25.6.2.2 [istream.iterator.cons] as indicated:

```
istream_iterator(const istream_iterator& x) = default;
```

-5- *Postconditions:* `in_stream == x.in_stream` is `true`.

~~-6- *Remarks:* If `is_trivially_copy_constructible_v<T>` is `true`, then this constructor is trivial.~~

[2021-09-30; Jonathan revises wording after reflector discussion]

A benefit of triviality is that it is constexpr, want to preserve that.

[2021-10-14; Reflector poll]

Set priority to 3 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to N4892.

1. Modify the class synopsis in 25.6.2.1 [istream.iterator.general] as indicated:

```
constexpr istream_iterator();
constexpr istream_iterator(default_sentinel_t);
istream_iterator(istream_type& s);
constexpr istream_iterator(const istream_iterator& x) = default;
~istream_iterator() = default;
istream_iterator& operator=(const istream_iterator&) = default;
```

2. Modify 25.6.2.2 [istream.iterator.cons] as indicated:

```
constexpr istream_iterator(const istream_iterator& x) = default;
```

-5- *Postconditions:* `in_stream == x.in_stream` is `true`.

-6- *Remarks:* ~~If `is_trivially_copy_constructible_v<T>` is `true`, then this constructor is trivial.~~ If the initializer `T(x.value)` in the declaration `auto val = T(x.value);` is a constant initializer ([expr.const]), then this constructor is a constexpr constructor.

[2022-10-12; Jonathan provides improved wording]

Discussed on the reflector September 2021.

[2022-10-13; Jonathan revises wording to add a noexcept-specifier]

[2022-11-07; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify the class synopsis in 25.6.2.1 [istream.iterator.general] as indicated:

```
constexpr istream_iterator();
constexpr istream_iterator(default_sentinel_t);
istream_iterator(istream_type& s);
constexpr istream_iterator(const istream_iterator& x) noexcept(see below) = default;
~istream_iterator() = default;
istream_iterator& operator=(const istream_iterator&) = default;
```

2. Modify 25.6.2.2 [istream.iterator.cons] as indicated:

```
constexpr istream_iterator(const istream_iterator& x) noexcept(see below) = default;
```

~~-5- *Postconditions:* `in_stream == x.in_stream` is `true`.~~

-?- *Effects:* Initializes `in_stream` with `x.in_stream` and initializes `value` with `x.value`.

-6- *Remarks:* ~~If `is_trivially_copy_constructible_v<T>` is `true`, then this constructor is trivial.~~ An invocation of this constructor may be used in a core constant expression if and only if the initialization of `value` from `x.value` is a constant subexpression ([defns.const.subexpr]). The exception specification is equivalent to `is_nothrow_copy_constructible_v<T>`.

3629. `make_error_code` and `make_error_condition` are customization points

Section: 19.5 [syserr] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2021-10-31 **Last modified:** 2022-09-23

Priority: 2

View all other issues in [syserr].

Discussion:

The rule in 16.4.2.2 [contents] means that the calls to `make_error_code` in 19.5.4.2 [syserr.errcode.constructors] and 19.5.4.3 [syserr.errcode.modifiers] are required to call `std::make_error_code`, which means program-defined error codes do not work. The same applies to the `make_error_condition` calls in 19.5.5.2 [syserr.errcondition.constructors] and 19.5.5.3 [syserr.errcondition.modifiers].

They need to use ADL. This is what all known implementations (including Boost.System) do.

[2022-01-29; Reflector poll]

Set priority to 2 after reflector poll.

Previous resolution [SUPERSEDED]:

> This wording is relative to N4901.
>
> 1. Modify 19.5.2 [system.error.syn] as indicated:
>
> > -1- The value of each `enum errc` constant shall be the same as the value of the `<cerrno>` macro shown in the above synopsis. Whether or not the `<system_error>` implementation exposes the `<cerrno>` macros is unspecified.
> >
> > -?- Invocations of `make_error_code` and `make_error_condition` shown in subclause 19.5 [syserr] select a function to call via overload resolution (12.2 [over.match]) on a candidate set that includes the lookup set found by argument dependent lookup (6.5.4 [basic.lookup.argdep]).
> >
> > -2- The `is_error_code_enum` and `is_error_condition_enum` templates may be specialized for program-defined types to indicate that such types are eligible for `class error_code` and `class error_condition` implicit conversions, respectively.
> >
> > *[Note 1:* Conversions from such types are done by program-defined overloads of `make_error_code` and `make_error_condition`, found by ADL. *—end note]*

[2022-08-25; Jonathan Wakely provides improved wording]

Discussed in LWG telecon and decided on new direction:

- Add `make_error_code` and `make_error_condition` to 16.4.2.2 [contents] as done for `swap`. Describe form of lookup used for them.
- Respecify `error_code` and `error_condition` constructors in terms of "*Effects*: Equivalent to" so that the requirements on program-defined overloads found by ADL are implied by those effects.

[2022-09-07; Jonathan Wakely revises wording]

Discussed in LWG telecon. Decided to change "established as-if by performing unqualified name lookup and argument-dependent lookup" to simply "established as-if by performing argument-dependent lookup".

This resolves the question of whether `std::make_error_code(errc)`, `std::make_error_code(io_errc)`, etc. should be visible to the unqualified name lookup. This affects whether a program-defined type that specializes `is_error_code_enum` but doesn't provide an overload of `make_error_code` should find the overloads in namespace `std` and consider them for overload resolution, via implicit conversion to `std::errc`, `std::io_errc`, etc.

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4910.

- Modify 16.4.2.2 [contents] as indicated:

> -3- Whenever an unqualified name other than `swap`, `make_error_code`, or `make_error_condition` is used in the specification of a declaration `D` in 17 [support] through 33 [thread] or D [depr], its meaning is established as-if by performing unqualified name lookup (6.5.3 [basic.lookup.unqual]) in the context of `D`.
>
> [*Note 1*: Argument-dependent lookup is not performed. — *end note*]
>
> Similarly, the meaning of a *qualified-id* is established as-if by performing qualified name lookup (6.5.5 [basic.lookup.qual]) in the context of `D`.
>
> [*Example 1*: The reference to `is_array_v` in the specification of `std::to_array` (24.3.7.6 [array.creation]) refers to `::std::is_array_v`. — *end example*]
>
> [*Note 2*: Operators in expressions (12.2.2.3 [over.match.oper]) are not so constrained; see 16.4.6.4 [global.functions]. — *end note*]
>
> The meaning of the unqualified name `swap` is established in an overload resolution context for swappable values (16.4.4.3 [swappable.requirements]). The meanings of the unqualified names `make_error_code` and `make_error_condition` are established as-if by performing argument-dependent lookup (6.5.4 [basic.lookup.argdep]).

- Modify 19.5.4.2 [syserr.errcode.constructors] as indicated:

```
error_code() noexcept;
```

> ~~-1- *Postconditions:* `val_ == 0` and `cat_ == &system_category()`.~~
> -1- *Effects*: Initializes `val_` with `0` and `cat_` with `&system_category()`.

```
error_code(int val, const error_category& cat) noexcept;
```

~~-2- *Postconditions*: `val_ == val` and `cat_ == &cat`.~~
-2- *Effects*: Initializes `val_` with `val` and `cat_` with `&cat`.

```
template<class ErrorCodeEnum>
  error_code(ErrorCodeEnum e) noexcept;
```

-3- *Constraints*: `is_error_code_enum_v<ErrorCodeEnum>` is `true`.

~~-4- *Postconditions*: `*this == make_error_code(e)`.~~
-4- *Effects*: Equivalent to:

```
error_code ec = make_error_code(e);
assign(ec.value(), ec.category());
```

- Modify 19.5.4.3 [syserr.errcode.modifiers] as indicated:

```
template<class ErrorCodeEnum>
  error_code& operator=(ErrorCodeEnum e) noexcept;
```

-2- *Constraints*: `is_error_code_enum_v<ErrorCodeEnum>` is `true`.

~~-3- *Postconditions*: `*this == make_error_code(e)`.~~
-3- *Effects*: Equivalent to:

```
error_code ec = make_error_code(e);
assign(ec.value(), ec.category());
```

-4- *Returns*: `*this`.

- Modify 19.5.5.2 [syserr.errcondition.constructors] as indicated:

```
error_condition() noexcept;
```

~~-1- *Postconditions*: `val_ == 0` and `cat_ == &generic_category()`.~~
-1- *Effects*: Initializes `val_` with `0` and `cat_` with `&generic_category()`.

```
error_condition(int val, const error_category& cat) noexcept;
```

~~-2- *Postconditions*: `val_ == val` and `cat_ == &cat`.~~
-2- *Effects*: Initializes `val_` with `val` and `cat_` with `&cat`.

```
template<class ErrorConditionEnum>
  error_condition(ErrorConditionEnum e) noexcept;
```

-3- *Constraints*: `is_error_condition_enum_v<ErrorConditionEnum>` is `true`.

~~-4- *Postconditions*: `*this == make_error_condition(e)`.~~
-4- *Effects*: Equivalent to:

```
error_condition ec = make_error_condition(e);
assign(ec.value(), ec.category());
```

- Modify 19.5.5.3 [syserr.errcondition.modifiers] as indicated:

```
template<class ErrorConditionEnum>
  error_condition& operator=(ErrorConditionEnum e) noexcept;
```

-2- *Constraints*: `is_error_condition_enum_v<ErrorConditionEnum>` is `true`.

~~-3- *Postconditions*: `*this == make_error_condition(e)`.~~
-3- *Effects*: Equivalent to:

```
error_condition ec = make_error_condition(e);
assign(ec.value(), ec.category());
```

-4- *Returns*: `*this`.

3646. `std::ranges::view_interface::size` returns a signed type

Section: 26.5.3.1 [view.interface.general] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2021-11-29 **Last modified:** 2022-09-23

Priority: 3

View other active issues in [view.interface.general].

View all other issues in [view.interface.general].

Discussion:

According to 26.5.3.1 [view.interface.general], `view_interface::size` returns the difference between the sentinel and the beginning iterator, which always has a signed-integer-like type. However, IIUC the decision that a `size` member function should return an unsigned type by default was made when adopting P1227R2, and the relative changes of the ranges library were done in P1523R1. I don't know why `view_interface::size` was unchanged, while `ranges::size` returns an unsigned type in similar situations (26.3.10 [range.prim.size] (2.5)).

If we want to change `views_interface::size` to return an unsigned type, the both overloads should be changed as below:

```
constexpr auto size() requires forward_range<D> &&
  sized_sentinel_for<sentinel_t<D>, iterator_t<D>> {
    return to-unsigned-like(ranges::end(derived()) - ranges::begin(derived()));
  }
constexpr auto size() const requires forward_range<const D> &&
  sized_sentinel_for<sentinel_t<const D>, iterator_t<const D>> {
    return to-unsigned-like(ranges::end(derived()) - ranges::begin(derived()));
  }
```

[2022-01-30; Reflector poll]

Set priority to 3 after reflector poll.

[2022-06-22; Reflector poll]

LEWG poll approved the proposed resolution

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll in July 2022.

Proposed resolution:

This wording is relative to N4901.

1. Modify 26.5.3.1 [view.interface.general], class template `view_interface` synopsis, as indicated:

```
[…]
constexpr auto size() requires forward_range<D> &&
  sized_sentinel_for<sentinel_t<D>, iterator_t<D>> {
    return to-unsigned-like(ranges::end(derived()) - ranges::begin(derived()));
  }
constexpr auto size() const requires forward_range<const D> &&
  sized_sentinel_for<sentinel_t<const D>, iterator_t<const D>> {
    return to-unsigned-like(ranges::end(derived()) - ranges::begin(derived()));
  }
[…]
```

3677. Is a *cv*-qualified `pair` specially handled in uses-allocator construction?

Section: 20.2.8.2 [allocator.uses.construction] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2022-02-16 **Last modified:** 2022-10-06

Priority: 2

View other active issues in [allocator.uses.construction].

View all other issues in [allocator.uses.construction].

Discussion:

It seems unclear whether *cv*-qualified `pair` specializations are considered as specializations of `pair` in 20.2.8.2 [allocator.uses.construction].

Currently MSVC STL only considered *cv*-unqualified pair types as such specializations, while libstdc++ accept both *cv*-unqualified and `const`-qualified pair types as such specializations. The resolution of LWG 3525 uses `remove_cv_t`, which possibly imply that the specialization of pair may be *cv*-qualified.

The difference can be observed via the following program:

```
#include <utility>
#include <memory>
#include <vector>
#include <cassert>

template<class T>
class payload_ator {

  int payload{};

public:
  payload_ator() = default;

  constexpr explicit payload_ator(int n) noexcept : payload{n} {}
```

```
  template<class U>
  constexpr explicit payload_ator(payload_ator<U> a) noexcept : payload{a.payload} {}

  friend bool operator==(payload_ator, payload_ator) = default;

  template<class U>
  friend constexpr bool operator==(payload_ator x, payload_ator<U> y) noexcept
  {
    return x.payload == y.payload;
  }

  using value_type = T;

  constexpr T* allocate(std::size_t n) { return std::allocator<T>{}.allocate(n); }

  constexpr void deallocate(T* p, std::size_t n) { return std::allocator<T>{}.deallocate(p, n); }

  constexpr int get_payload() const noexcept { return payload; }
};

bool test()
{
  constexpr int in_v = 42;
  using my_pair_t = std::pair<int, std::vector<int, payload_ator<int>>>;
  auto out_v = std::make_obj_using_allocator<const my_pair_t>(payload_ator<int>{in_v}).second.get_allocator().get_payload();
  return in_v == out_v;
}

int main()
{
  assert(test()); // passes only if a const-qualified pair specialization is considered as a pair specialization
}
```

[2022-03-04; Reflector poll]

Set priority to 2 after reflector poll.

[2022-08-24; LWG telecon]

Change every `T` to `remove_cv_t<T>`.

[2022-08-25; Jonathan Wakely provides wording]

Previous resolution [SUPERSEDED]:

> This wording is relative to N4910.
>
> - Modify 20.2.8.2 [allocator.uses.construction] as indicated, using `remove_cv_t` in every *Constraints*: element:
>
> *Constraints*: `remove_cv_t<T>` [is|is not] a specialization of `pair`

[2022-09-23; Jonathan provides improved wording]

[2022-09-30; moved to Tentatively Ready after seven votes in reflector poll]

Proposed resolution:

This wording is relative to N4917.

1. Modify 20.2.8.2 [allocator.uses.construction] as indicated, using `remove_cv_t` in every *Constraints*: element (paragraphs 4, 6, 8, 10, 12, 14, 17):

   Constraints: `remove_cv_t<T>` [is|is not] a specialization of `pair`

2. Add `remove_cv_t` in paragraph 5:

   -5- *Returns*: A `tuple` value determined as follows:

   (5.1) — If `uses_allocator_v<remove_cv_t<T>, Alloc>` is `false` and `is_constructible_v<T, Args...>` is `true`, return `forward_as_tuple(std::forward<Args>(args)...)`.

   (5.2) — Otherwise, if `uses_allocator_v<remove_cv_t<T>, Alloc>` is `true` and `is_constructible_v<T, allocator_arg_t, const Alloc&, Args...>` is `true`, return

   ```
   tuple<allocator_arg_t, const Alloc&, Args&&...>(
     allocator_arg, alloc, std::forward<Args>(args)...)
   ```

   (5.3) — Otherwise, if `uses_allocator_v<remove_cv_t<T>, Alloc>` is `true` and `is_constructible_v<T, Args..., const Alloc&>` is `true`, return `forward_as_tuple(std::forward<Args>(args)..., alloc)`.

   (5.4) — Otherwise, the program is ill-formed.

3. Rephrase paragraph 7 in terms of the pair member types:

-?- Let `T1` be `T::first_type`. Let `T2` be `T::second_type`.

-6- *Constraints*: `remove_cv_t<T>` is a specialization of `pair`

-7- *Effects*:: ~~For `I` specified as `pair<T1, T2>`, equivalent~~ Equivalent to:

3732. `prepend_range` and `append_range` can't be amortized constant time

Section: 24.2.4 [sequence.reqmts] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2022-07-06 **Last modified:** 2022-08-31

Priority: Not Prioritized

View other active issues in [sequence.reqmts].

View all other issues in [sequence.reqmts].

Discussion:

24.2.4 [sequence.reqmts]/69 says "An implementation shall implement them so as to take amortized constant time." followed by a list of operations that includes the newly added `append_range` and `prepend_range`. Obviously these operations cannot be implemented in amortized constant time.

Because the actual complexity of these operations are already specified in the concrete container specification, we can just exclude them here.

[2022-08-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4910.

1. Modify 24.2.4 [sequence.reqmts] as indicated:

-69- The following operations are provided for some types of sequence containers but not others. ~~An implementation shall implement them~~Operations other than `prepend_range` and `append_range` are implemented so as to take amortized constant time.

3736. `move_iterator` missing `disable_sized_sentinel_for` specialization

Section: 25.2 [iterator.synopsis] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-07-14 **Last modified:** 2022-07-16

Priority: Not Prioritized

View other active issues in [iterator.synopsis].

View all other issues in [iterator.synopsis].

Discussion:

Since `reverse_iterator::operator-` is not constrained, the standard adds a `disable_sized_sentinel_for` specialization for it to avoid situations where the underlying iterator can be subtracted making `reverse_iterator` accidentally model `sized_sentinel_for`.

However, given that `move_iterator::operator-` is also unconstrained and the standard does not have the `disable_sized_sentinel_for` specialization for it, this makes `subrange<move_iterator<I>, move_iterator<I>>` unexpectedly satisfy `sized_range` and incorrectly use `I::operator-` to get size when `I` can be subtracted but not modeled `sized_sentinel_for<I>`.

In addition, since P2520 makes `move_iterator` no longer just `input_iterator`, `ranges::size` can get the size of the range by subtracting two `move_iterator` pairs, this also makes `r | views::as_rvalue` may satisfy `sized_range` when neither `r` nor `r | views::reverse` is `sized_range`.

We should add a `move_iterator` version of the `disable_sized_sentinel_for` specialization to the standard to avoid the above situation.

[2022-08-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

"but I don't think the issue text is quite right - both `move_iterator` and `reverse_iterator`'s `operator-` are constrained on `x.base() - y.base()` being valid."

Does anyone remember why we did this for `reverse_iterator` and not `move_iterator`?

Proposed resolution:

This wording is relative to N4910.

1. Modify 25.2 [iterator.synopsis], header `<iterator>` synopsis, as indicated:

```
namespace std::ranges {
  […]
  template<class Iterator>
    constexpr move_iterator<Iterator> make_move_iterator(Iterator i);

  template<class Iterator1, class Iterator2>
      requires (!sized_sentinel_for<Iterator1, Iterator2>)
    inline constexpr bool disable_sized_sentinel_for<move_iterator<Iterator1>,
                                                     move_iterator<Iterator2>> = true;

  […]
}
```

3738. Missing preconditions for `take_view` constructor

Section: 26.7.10.2 [range.take.view] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-07-15 **Last modified:** 2022-07-16

Priority: Not Prioritized

View all other issues in [range.take.view].

Discussion:

When `V` does not model `sized_range`, `take_view::begin` returns `counted_iterator(ranges::begin(base_), count_)`. Since the `counted_iterator` constructor (25.5.7.2 [counted.iter.const]) already has a precondition that `n >= 0`, we should add this to `take_view` as well, which is consistent with `drop_view`.

[2022-08-23; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4910.

1. Modify 26.7.10.2 [range.take.view] as indicated:

```
constexpr take_view(V base, range_difference_t<V> count);
```

-?- *Preconditions*: `count >= 0` is `true`.

-1- *Effects*: Initializes *base_* with `std::move(base)` and *count_* with `count`.

3743. `ranges::to`'s `reserve` may be ill-formed

Section: 26.5.7.2 [range.utility.conv.to] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-07-21 **Last modified:** 2022-07-24

Priority: Not Prioritized

View other active issues in [range.utility.conv.to].

View all other issues in [range.utility.conv.to].

Discussion:

When the "reserve" branch is satisfied, `ranges::to` directly passes the result of `ranges::size(r)` into the `reserve` call. However, given that the standard only guarantees that integer-class type can be explicitly converted to any integer-like type (25.3.4.4 [iterator.concept.winc] p6), this makes the call potentially ill-formed, since `ranges::size(r)` may return an integer-class type:

```
#include <ranges>
#include <vector>

int main() {
  auto r = std::ranges::subrange(std::views::iota(0ULL) | std::views::take(5), 5);
  auto v = r | std::ranges::to<std::vector<std::size_t>>(0); // cannot implicitly convert _Unsigned128 to size_t in MSVC-STL
}
```

We should do an explicit cast before calling `reserve`.

[2022-08-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Are we all happy that the result of conversion to the container's size type may be less than the length of the source range, so the reservation is too small but we don't realize until pushing the max_size() + 1st element fails? I think it's acceptable that converting pathologically large ranges to containers fails kind

of messily, but I could imagine throwing if the range length is greater than container's max_size().

Proposed resolution:

This wording is relative to N4910.

1. Modify 26.5.7.2 [range.utility.conv.to] as indicated:

```
template<class C, input_range R, class... Args> requires (!view<C>)
  constexpr C to(R&& r, Args&&... args);
```

-1- *Returns*: An object of type `C` constructed from the elements of `r` in the following manner:

(1.1) — If `convertible_to<range_reference_t<R>, range_value_t<C>>` is `true`:

(1.1.1) — If `constructible_from<C, R, Args...>` is `true`:

```
C(std::forward<R>(r), std::forward<Args>(args)...)
```

(1.1.2) — Otherwise, if `constructible_from<C, from_range_t, R, Args...>` is `true`:

```
C(from_range, std::forward<R>(r), std::forward<Args>(args)...)
```

(1.1.3) — Otherwise, if

(1.1.3.1) — `common_range<R>` is `true`,

(1.1.3.2) — *`cpp17-input-iterator`*`<iterator_t<R>>` is `true`, and

(1.1.3.3) — `constructible_from<C, iterator_t<R>, sentinel_t<R>, Args...>` is `true`:

```
C(ranges::begin(r), ranges::end(r), std::forward<Args>(args)...)
```

(1.1.4) — Otherwise, if

(1.1.4.1) — `constructible_from<C, Args...>` is `true`, and

(1.1.4.2) — *`container-insertable`*`<C, range_reference_t<R>>` is `true`:

```
C c(std::forward<Args>(args)...);
if constexpr (sized_range<R> && reservable-container<C>)
  c.reserve(static cast<range size t<C>>(ranges::size(r)));
ranges::copy(r, container-inserter<range_reference_t<R>>(c));
```

(1.2) — Otherwise, if `input_range<range_reference_t<R>>` is `true`:

```
to<C>(r | views::transform([](auto&& elem) {
  return to<range_value_t<C>>(std::forward<decltype(elem)>(elem));
}), std::forward<Args>(args)...);
```

(1.3) — Otherwise, the program is ill-formed.

3745. `std::atomic_wait` and its friends lack `noexcept`

Section: 33.5.2 [atomics.syn] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2022-07-25 **Last modified:** 2022-07-30

Priority: Not Prioritized

View other active issues in [atomics.syn].

View all other issues in [atomics.syn].

Discussion:

Currently function templates `std::atomic_wait`, `std::atomic_wait_explicit`, `std::atomic_notify_one`, and `std::atomic_notify_all` are not `noexcept` in the Working Draft, but the equivalent member functions are all `noexcept`. I think these function templates should be specified as `noexcept`, in order to be consistent with the `std::atomic_flag_*` free functions, the corresponding member functions, and other `std::atomic_*` function templates.

Mainstream implementations (libc++, libstdc++, and MSVC STL) have already added `noexcept` to them.

[2022-07-30; Daniel provides wording]

[2022-08-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

"Technically there's a difference between these and the member functions - the pointer can be null - but we don't seem to have let that stop us from adding noexcept to the rest of these functions."

Proposed resolution:

This wording is relative to N4910.

1. Modify 33.5.2 [atomics.syn], header <atomic> synopsis, as indicated:

```
[…]
template<class T>
  void atomic_wait(const volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
  void atomic_wait(const atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
  void atomic_wait_explicit(const volatile atomic<T>*, typename atomic<T>::value_type,
                            memory_order) noexcept;
template<class T>
  void atomic_wait_explicit(const atomic<T>*, typename atomic<T>::value_type,
                            memory_order) noexcept;
template<class T>
  void atomic_notify_one(volatile atomic<T>*) noexcept;
template<class T>
  void atomic_notify_one(atomic<T>*) noexcept;
template<class T>
  void atomic_notify_all(volatile atomic<T>*) noexcept;
template<class T>
  void atomic_notify_all(atomic<T>*) noexcept;
[…]
```

3746. `optional`'s spaceship with `U` with a type derived from `optional` causes infinite constraint meta-recursion

Section: 22.5.8 [optional.comp.with.t] **Status:** Tentatively Ready **Submitter:** Ville Voutilainen **Opened:** 2022-07-25 **Last modified:** 2022-08-23

Priority: Not Prioritized

View all other issues in [optional.comp.with.t].

Discussion:

What ends up happening is that the constraints of `operator<=>(const optional<T>&, const U&)` end up in `three_way_comparable_with`, and then in *`partially-ordered-with`*, and the expressions there end up performing a conversion from `U` to an `optional`, and we end up instantiating the same `operator<=>` again, evaluating its constraints again, until the compiler bails out.

See an online example here.

All implementations end up with infinite meta-recursion.

The solution to the problem is to stop the meta-recursion by constraining the spaceship with `U` so that `U` is not publicly and unambiguously derived from a specialization of optional, SFINAEing that candidate out, and letting 22.5.6 [optional.relops]/20 perform the comparison instead.

[2022-08-23; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4910.

> [*Drafting note:* This proposed wording removes the only use of *`is-optional`*.]

1. Modify 22.5.2 [optional.syn], header <optional> synopsis, as indicated:

```
[…]
namespace std {
  // 22.5.3 [optional.optional], class template optional
  template<class T>
    class optional;

  template<class T>
    constexpr bool is-optional = false; // exposition only
  template<class T>
    constexpr bool is-optional<optional<T>> = true; // exposition only
  template<class T>
    concept is-derived-from-optional = requires(const T& t) { // exposition only
      []<class U>(const optional<U>&){ }(t);
    };
  […]
  // 22.5.8 [optional.comp.with.t], comparison with T
  […]
  template<class T, class U> constexpr bool operator>=(const T&, const optional<U>&);
  template<class T, class U> requires (!is-derived-from-optional<U>) && three_way_comparable_with<T, U>
```

```
    constexpr compare_three_way_result_t<T, U>
      operator<=>(const optional<T>&, const U&);
  […]
}
```

2. Modify 22.5.8 [optional.comp.with.t] as indicated:

```
template<class T, class U> requires (!is-derived-from-optional<U>) && three_way_comparable_with<T, U>
  constexpr compare_three_way_result_t<T, U>
    operator<=>(const optional<T>&, const U&);
```

-25- *Effects:* Equivalent to: `return x.has_value() ? *x <=> v : strong_ordering::less;`

3747. `ranges::uninitialized_copy_n`, `ranges::uninitialized_move_n`, and `ranges::destroy_n` should use `std::move`

Section: 27.11.5 [uninitialized.copy], 27.11.6 [uninitialized.move], 27.11.9 [specialized.destroy] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-07-28 **Last modified:** 2022-08-23

Priority: Not Prioritized

View all other issues in [uninitialized.copy].

Discussion:

Currently, `ranges::uninitialized_copy_n` has the following equivalent *Effects*:

```
auto t = uninitialized_copy(counted_iterator(ifirst, n),
                            default_sentinel, ofirst, olast);
return {std::move(t.in).base(), t.out};
```

Given that `ifirst` is just an `input_iterator` which is not guaranteed to be `copyable`, we should move it into `counted_iterator`. The same goes for `ranges::uninitialized_move_n` and `ranges::destroy_n`.

[2022-08-23; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4910.

1. Modify 27.11.5 [uninitialized.copy] as indicated:

```
namespace ranges {
  template<input_iterator I, nothrow-forward-iterator O, nothrow-sentinel-for<O> S>
    requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
    uninitialized_copy_n_result<I, O>
      uninitialized_copy_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast);
}
```

-9- *Preconditions*: [`ofirst, olast`) does not overlap with `ifirst + [0, n)`.

-10- *Effects*: Equivalent to:

```
auto t = uninitialized_copy(counted_iterator(std::move(ifirst), n),
                            default_sentinel, ofirst, olast);
return {std::move(t.in).base(), t.out};
```

2. Modify 27.11.6 [uninitialized.move] as indicated:

```
namespace ranges {
  template<input_iterator I, nothrow-forward-iterator O, nothrow-sentinel-for<O> S>
    requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
    uninitialized_move_n_result<I, O>
      uninitialized_move_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast);
}
```

-8- *Preconditions*: [`ofirst, olast`) does not overlap with `ifirst + [0, n)`.

-9- *Effects*: Equivalent to:

```
auto t = uninitialized_move(counted_iterator(std::move(ifirst), n),
                            default_sentinel, ofirst, olast);
return {std::move(t.in).base(), t.out};
```

3. Modify 27.11.9 [specialized.destroy] as indicated:

```
namespace ranges {
  template<nothrow-input-iterator I>
    requires destructible<iter_value_t<I>>
```

```
    constexpr I destroy_n(I first, iter_difference_t<I> n) noexcept;
}
```

-5- *Effects*: Equivalent to:

```
return destroy(counted_iterator(std::move(first), n), default_sentinel).base();
```

3750. Too many papers bump `__cpp_lib_format`

Section: 17.3.2 [version.syn] **Status:** Tentatively Ready **Submitter:** Barry Revzin **Opened:** 2022-08-04 **Last modified:** 2022-08-23

Priority: Not Prioritized

View other active issues in [version.syn].

View all other issues in [version.syn].

Discussion:

As pointed out by Casey Carter, four papers approved at the recent July 2022 plenary:

- P2419R2 "Clarify handling of encodings in localized formatting of chrono types"
- P2508R1 "Expose `std::basic-format-string<charT, Args...>`"
- P2286R8 "Formatting Ranges"
- P2585R1 "Improve container default formatting"

all bump the value of `__cpp_lib_format`. We never accounted for all of these papers being moved at the same time, and these papers have fairly different implementation complexities.

Victor Zverovich suggests that we instead add `__cpp_lib_format_ranges` (with value `202207L`) for the two formatting ranges papers (P2286 and P2585, which should probably be implemented concurrently anyway, since the latter modifies the former) and `bump __cpp_lib_format` for the other two, which are both minor changes.

We should do that.

[2022-08-23; Reflector poll]

Set status to Tentatively Ready after 12 votes in favour during reflector poll.

Proposed resolution:

1. Modify 17.3.2 [version.syn] as indicated:

```
#define __cpp_lib_format          202207L // also in <format>
#define __cpp_lib_format_ranges   202207L // also in <format>
```

3751. Missing feature macro for `flat_set`

Section: 17.3.2 [version.syn] **Status:** Tentatively Ready **Submitter:** Barry Revzin **Opened:** 2022-08-04 **Last modified:** 2022-08-23

Priority: Not Prioritized

View other active issues in [version.syn].

View all other issues in [version.syn].

Discussion:

As pointed out by Casey Carter, while there is a feature macro for `flat_map` in P0429, there is no corresponding macro for flat_set in P1222. We should add one.

[2022-08-23; Reflector poll]

Set status to Tentatively Ready after 10 votes in favour during reflector poll.

Proposed resolution:

1. Modify 17.3.2 [version.syn] as indicated:

```
#define __cpp_lib_flat_map  202207L // also in <flat_map>
#define __cpp_lib_flat_set  202207L // also in <flat_set>
```

3755. *tuple-for-each* can call user-defined operator,

Section: 26.7.23.2 [range.zip.view] **Status:** Tentatively Ready **Submitter:** Nicole Mazzuca **Opened:** 2022-08-26 **Last modified:** 2022-09-25

Priority: Not Prioritized

View other active issues in [range.zip.view].

View all other issues in [range.zip.view].

Discussion:

The specification for *tuple-for-each* is:

```
template<class F, class Tuple>
constexpr auto tuple-for-each(F&& f, Tuple&& t) { // exposition only
  apply([&]<class... Ts>(Ts&&... elements) {
    (invoke(f, std::forward<Ts>(elements)), ...);
  }, std::forward<Tuple>(t));
}
```

Given

```
struct Evil {
  void operator,(Evil) {
        abort();
    }
};
```

and `tuple<int, int> t`, then *tuple-for-each*`([](int) { return Evil{}; }, t)`, the program will (unintentionally) abort.

It seems likely that our `Evil`'s `operator,` should not be called.

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Feedback from reviewers:

> "NAD. This exposition-only facility is only used with things that return `void`. As far as I know, users can't define `operator,` for `void`." "If I see the `void` cast, I don't need to audit the uses or be concerned that we'll add a broken use in the future."

Proposed resolution:

This wording is relative to the forthcoming C++23 CD.

- Modify 26.7.5 [range.adaptor.tuple] as indicated:

```
template<class F, class Tuple>
constexpr auto tuple-for-each(F&& f, Tuple&& t) { // exposition only
  apply([&]<class... Ts>(Ts&&... elements) {
    (static_cast<void>(invoke(f, std::forward<Ts>(elements))), ...);
  }, std::forward<Tuple>(t));
}
```

3757. What's the effect of `std::forward_like<void>(x)`?

Section: 22.2.4 [forward] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2022-08-24 **Last modified:** 2022-09-23

Priority: Not Prioritized

View all other issues in [forward].

Discussion:

Currently the return type of `std::forward_like` is specified by the following bullet:

> — Let `V` be
>
> ```
> OVERRIDE_REF(T&&, COPY_CONST(remove_reference_t<T>, remove_reference_t<U>))
> ```

where `T&&` is not always valid, e.g. it's invalid when `T` is `void`.

A strait forward reading may suggest that there is a hard error when `T` is not referenceable (which is currently implemented in MSVC STL), but this seems not clarified. It is unclear to me whether the intent is that hard error, substitution failure, or no error is caused when `T&&` is invalid.

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 22.2.4 [forward] as indicated:

```
template<class T, class U>
  [[nodiscard]] constexpr auto forward_like(U&& x) noexcept -> see below;
```

Mandates: `T` is a referenceable type (3.46 [defns.referenceable]).

[…]

3759. `ranges::rotate_copy` should use `std::move`

Section: 27.7.11 [alg.rotate] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-08-25 **Last modified:** 2022-09-23

Priority: Not Prioritized

View all other issues in [alg.rotate].

Discussion:

The range version of `ranges::rotate_copy` directly passes the `result` to the iterator-pair version. Since the type of `result` only models `weakly_incrementable` and may not be copied, we should use `std::move` here.

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4910.

1. Modify 27.7.11 [alg.rotate] as indicated:

```
template<forward_range R, weakly_incrementable O>
  requires indirectly_copyable<iterator_t<R>, O>
  constexpr ranges::rotate_copy_result<borrowed_iterator_t<R>, O>
    ranges::rotate_copy(R&& r, iterator_t<R> middle, O result);
```

-11- *Effects*: Equivalent to:

```
return ranges::rotate_copy(ranges::begin(r), middle, ranges::end(r), std::move(result));
```

3760. `cartesian_product_view::iterator`'s `parent_` is never valid

Section: 26.7.31 [range.cartesian] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-08-27 **Last modified:** 2022-09-23

Priority: Not Prioritized

Discussion:

`cartesian_product_view::iterator` has a pointer member `parent_` that points to `cartesian_product_view`, but its constructor only accepts a `tuple` of iterators, which makes `parent_` always default-initialized to `nullptr`.

The proposed resolution is to add an aliased `Parent` parameter to the constructor and initialize `parent_` with `addressof`, as we usually do.

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.7.31.2 [range.cartesian.view] as indicated:

```
constexpr iterator<false> begin()
  requires (!simple-view<First> || ... || !simple-view<Vs>);
```

-2- *Effects*: Equivalent to: `return iterator<false>(*this, tuple-transform(ranges::begin, bases_));`

```
constexpr iterator<true> begin() const
  requires (range<const First> && ... && range<const Vs>);
```

-3- *Effects*: Equivalent to: `return iterator<true>(*this, tuple-transform(ranges::begin, bases_));`

```
constexpr iterator<false> end()
  requires ((!simple-view<First> || ... || !simple-view<Vs>)
    && cartesian-product-is-common<First, Vs...>);
constexpr iterator<true> end() const
  requires cartesian-product-is-common<const First, const Vs...>;
```

-4- Let:

(4.1) — *is-const* be `true` for the const-qualified overload, and `false` otherwise;

(4.2) — *is-empty* be `true` if the expression `ranges::empty(rng)` is `true` for any `rng` among the underlying ranges except the first one and `false` otherwise; and

(4.3) — *begin-or-first-end*`(rng)` be expression-equivalent to *is-empty* `? ranges::begin(rng) :` *cartesian-common-arg-end*`(rng)` if `rng` is the first underlying range and `ranges::begin(rng)` otherwise.

-5- *Effects*: Equivalent to:

```
iterator<is-const> it(*this, tuple-transform(
  [](auto& rng){ return begin-or-first-end(rng); }, bases_));
return it;
```

2. Modify 26.7.31.3 [ranges.cartesian.iterator] as indicated:

```
namespace std::ranges {
  template<input_range First, forward_range... Vs>
    requires (view<First> && ... && view<Vs>)
  template<bool Const>
  class cartesian_product_view<First, Vs...>::iterator {
  public:
    […]

  private:
    using Parent = maybe-const<Const, cartesian_product_view>;          // exposition only
    Parent~~maybe-const<Const, cartesian_product_view>~~* parent_ = nullptr; // exposition only
    tuple<iterator_t<maybe-const<Const, First>>,
      iterator_t<maybe-const<Const, Vs>>...> current_;                  // exposition only

    template<size_t N = sizeof...(Vs)>
      constexpr void next();                                            // exposition only

    template<size_t N = sizeof...(Vs)>
      constexpr void prev();                                            // exposition only

    template<class Tuple>
      constexpr difference_type distance-from(Tuple t);                 // exposition only

    constexpr ~~explicit~~ iterator(Parent& parent, tuple<iterator_t<maybe-const<Const, First>>,
      iterator_t<maybe-const<Const, Vs>>...> current);                  // exposition only
  };
}
```

[…]

```
constexpr ~~explicit~~ iterator(Parent& parent, tuple<iterator_t<maybe-const<Const, First>>,
  iterator_t<maybe-const<Const, Vs>>...> current);
```

-10- *Effects*: Initializes *parent_* with `addressof(parent)` and *current_* with `std::move(current)`.

```
constexpr iterator(iterator<!Const> i) requires Const &&
  (convertible_to<iterator_t<First>, iterator_t<const First>> &&
    ... && convertible_to<iterator_t<Vs>, iterator_t<const Vs>>);
```

-11- *Effects*: Initializes *parent_* with `i.`*parent_* and *current_* with `std::move(i.`*current_*`)`.

3761. `cartesian_product_view::`*iterator*`::operator-` should pass by reference

Section: 26.7.31.3 [ranges.cartesian.iterator] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-08-27 **Last modified:** 2022-09-23

Priority: Not Prioritized

View other active issues in [ranges.cartesian.iterator].

View all other issues in [ranges.cartesian.iterator].

Discussion:

There are two problems with `cartesian_product_view::`*iterator*`::operator-`.

First, *distance-from* is not `const`-qualified, which would cause the common version of `operator-` to produce a hard error inside the function body since it invokes *distance-from* on a const *iterator* object.

Second, the non-common version of `operator-` passes *`iterator`* by value, which unnecessarily prohibits subtracting `default_sentinel` from an lvalue *`iterator`* whose first underlying iterator is non-`copyable`. Even if we `std::move` it into `operator-`, the other overload will still be ill-formed since it returns `-(i - s)` which will calls `i`'s deleted copy constructor.

The proposed resolution is to add `const` qualification to *`distance-from`* and make the non-common version of *`iterator`*`::operator-` pass by const reference. Although the `Tuple` parameter of *`distance-from`* is guaranteed to be `copyable`, I think it would be more appropriate to pass it by reference.

[2022-09-08]

As suggested by Tim Song, the originally submitted proposed wording has been refined to use `const Tuple&` instead of `Tuple&`.

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.7.31.3 [ranges.cartesian.iterator] as indicated:

```
namespace std::ranges {
  template<input_range First, forward_range... Vs>
    requires (view<First> && ... && view<Vs>)
  template<bool Const>
  class cartesian_product_view<First, Vs...>::iterator {
  public:
    […]

    friend constexpr difference_type operator-(const iterator& i, default_sentinel_t)
      requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;
    friend constexpr difference_type operator-(default_sentinel_t, const iterator& i)
      requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;

    […]
  private:
    […]

    template<class Tuple>
      constexpr difference_type distance-from(const Tuple& t) const;            // exposition only

    […]
  };
}
```

[…]

```
template<class Tuple>
  constexpr difference_type distance-from(const Tuple& t) const;
```

-7- Let:

(7.1) — *`scaled-size`*`(N)` be the product of `static_cast<difference_type>(ranges::size(std::get<N>(parent_->bases_)))` and *`scaled-size`*`(N + 1)` if `N < sizeof...(Vs)`, otherwise `static_cast<difference_type>(1)`;

(7.2) — *`scaled-distance`*`(N)` be the product of `static_cast<difference_type>(std::get<N>(current_) - std::get<N>(t))` and *`scaled-size`*`(N + 1)`; and

(7.3) — *`scaled-sum`* be the sum of *`scaled-distance`*`(N)` for every integer `0 ≤ N ≤ sizeof...(Vs)`.

-8- *Preconditions*: *`scaled-sum`* can be represented by `difference_type`.

-9- *Returns*: *`scaled-sum`*.

[…]

```
friend constexpr difference_type operator-(const iterator& i, default_sentinel_t)
  requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;
```

-32- Let *`end-tuple`* be an object of a type that is a specialization of `tuple`, such that:

(32.1) — `std::get<0>(`*`end-tuple`*`)` has the same value as `ranges::end(std::get<0>(i.`*`parent_->bases_`*`))`;

(32.2) — `std::get<`*`N`*`>(`*`end-tuple`*`)` has the same value as `ranges::begin(std::get<`*`N`*`>(i.`*`parent_->bases_`*`))` for every integer `1 ≤ N ≤ sizeof...(Vs)`.

-33- *Effects*: Equivalent to: `return i.`*`distance-from`*`(`*`end-tuple`*`);`

```
friend constexpr difference_type operator-(default_sentinel_t, const iterator& i)
  requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;
```

-34- *Effects*: Equivalent to: `return -(i - s);`

3762. `generator::iterator::operator==` should pass by reference

Section: 26.8.6 [coro.generator.iterator] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-08-27 **Last modified:** 2022-09-23

Priority: Not Prioritized

Discussion:

Currently, `generator::iterator::operator==` passes *`iterator`* by value. Given that *`iterator`* is a move-only type, this makes it impossible for `generator` to model `range`, since `default_sentinel` cannot be compared to the *`iterator`* of `const` lvalue and is not eligible to be its sentinel.

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.8.6 [coro.generator.iterator] as indicated:

```
namespace std {
  template<class Ref, class V, class Allocator>
  class generator<Ref, V, Allocator>::iterator {
  public:
    using value_type = value;
    using difference_type = ptrdiff_t;

    iterator(iterator&& other) noexcept;
    iterator& operator=(iterator&& other) noexcept;

    reference operator*() const noexcept(is_nothrow_copy_constructible_v<reference>);
    iterator& operator++();
    void operator++(int);

    friend bool operator==(const iterator& i, default_sentinel_t);

  private:
    coroutine_handle<promise_type> coroutine_; // exposition only
  };
}
```

[…]

```
friend bool operator==(const iterator& i, default_sentinel_t);
```

-10- *Effects*: Equivalent to: `return i.coroutine_.done();`

3764. `reference_wrapper::operator()` should propagate `noexcept`

Section: 22.10.6.5 [refwrap.invoke] **Status:** Tentatively Ready **Submitter:** Zhihao Yuan **Opened:** 2022-08-31 **Last modified:** 2022-09-25

Priority: Not Prioritized

View all other issues in [refwrap.invoke].

Discussion:

The following assertion does not hold, making the type less capable than the function pointers.

```
void f() noexcept;

std::reference_wrapper fn = f;
static_assert(std::is_nothrow_invocable_v<decltype(fn)>);
```

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Already implemented in MSVC and libstdc++.

Proposed resolution:

This wording is relative to N4910.

1. Modify 22.10.6.1 [refwrap.general], class template `reference_wrapper` synopsis, as indicated:

```
// 22.10.6.5 [refwrap.invoke], invocation
template<class... ArgTypes>
  constexpr invoke_result_t<T&, ArgTypes...> operator()(ArgTypes&&...) const
    noexcept(is_nothrow_invocable_v<T&, ArgTypes...>);
```

2. Modify 22.10.6.5 [refwrap.invoke] as indicated:

```
template<class... ArgTypes>
  constexpr invoke_result_t<T&, ArgTypes...>
    operator()(ArgTypes&&... args) const noexcept(is_nothrow_invocable_v<T&, ArgTypes...>);
```

-1- *Mandates*: `T` is a complete type.

-2- *Returns*: `INVOKE(get(), std::forward<ArgTypes>(args)...)`. (22.10.4 [func.require])

3765. `const_sentinel` should be constrained

Section: 25.2 [iterator.synopsis], 25.5.3.2 [const.iterators.alias] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-09-03 **Last modified:** 2022-09-23

Priority: Not Prioritized

View other active issues in [iterator.synopsis].

View all other issues in [iterator.synopsis].

Discussion:

The current standard does not have any constraints on `const_sentinel` template parameters, which makes it possible to pass almost any type of object to `make_const_sentinel`. It would be more appropriate to constrain the type to meet the minimum requirements of the sentinel type such as `semiregular` as `move_sentinel` does.

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 25.2 [iterator.synopsis], header `<iterator>` synopsis, as indicated:

```
#include <compare>              // see 17.12.1 [compare.syn]
#include <concepts>             // see 18.3 [concepts.syn]

namespace std {
  […]

  // 25.5.3 [const.iterators], constant iterators and sentinels
  // 25.5.3.2 [const.iterators.alias], alias templates
  […]
  template<input_iterator I>
    using const_iterator = see below;                                   // freestanding
  template<semiregular~~class~~ S>
    using const_sentinel = see below;                                   // freestanding

  […]
  template<input_iterator I>
    constexpr const_iterator<I> make_const_iterator(I it) { return it; }            // freestanding

  template<semiregular~~class~~ S>
    constexpr const_sentinel<S> make_const_sentinel(S s) { return s; }              // freestanding
  […]
}
```

2. Modify 25.5.3.2 [const.iterators.alias] as indicated:

```
template<semiregular~~class~~ S>
  using const_sentinel = see below;
```

-2- *Result*: If `S` models `input_iterator`, `const_iterator<S>`. Otherwise, `S`.

3770. `const_sentinel_t` is missing

Section: 26.2 [ranges.syn] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-09-05 **Last modified:** 2022-10-12

Priority: 3

View other active issues in [ranges.syn].

View all other issues in [ranges.syn].

Discussion:

The type alias pair `const_iterator` and `const_sentinel` and the factory function pair `make_const_iterator` and `make_const_sentinel` are defined in `<iterator>`, however, only `const_iterator_t` exists in `<ranges>`, which lacks `const_sentinel_t`, we should add it to ensure consistency.

The proposed resolution also lifts the type constraint of `const_iterator_t` to `input_range` to make it consistent with `const_iterator`, which already requires that its template parameter must model `input_iterator`.

[2022-09-23; Reflector poll]

Set priority to 3 after reflector poll.

Comment from a reviewer:

> "I don't see why we should redundantly constrain `const_iterator_t` with `input_range`; it's not as if we can make it more ill-formed or more SFINAE-friendly than it is now."

Previous resolution [SUPERSEDED]:

This wording is relative to N4917.

1. Modify 26.2 [ranges.syn], header `<ranges>` synopsis, as indicated:

```
#include <compare>              // see 17.12.1 [compare.syn]
#include <initializer_list>     // see 17.11.2 [initializer.list.syn]
#include <iterator>             // see 25.2 [iterator.synopsis]

namespace std::ranges {
  […]
  template<class T>
    using iterator_t = decltype(ranges::begin(declval<T&>()));                    // freestanding
  template<range R>
    using sentinel_t = decltype(ranges::end(declval<R&>()));                      // freestanding
  template<input_range R>
    using const_iterator_t = const_iterator<iterator_t<R>>;                       // freestanding
  template<range R>
    using const_sentinel_t = const_sentinel<sentinel_t<R>>;                       // freestanding
  […]
}
```

[2022-09-25; Daniel and Hewill provide alternative wording]

The alternative solution solely adds the suggested `const_sentinel_t` alias template and doesn't touch the `const_iterator_t` constraints.

[2022-10-12; Reflector poll]

Set status to "Tentatively Ready" after seven votes in favour.

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.2 [ranges.syn], header `<ranges>` synopsis, as indicated:

```
#include <compare>              // see 17.12.1 [compare.syn]
#include <initializer_list>     // see 17.11.2 [initializer.list.syn]
#include <iterator>             // see 25.2 [iterator.synopsis]

namespace std::ranges {
  […]
  template<class T>
    using iterator_t = decltype(ranges::begin(declval<T&>()));                    // freestanding
  template<range R>
    using sentinel_t = decltype(ranges::end(declval<R&>()));                      // freestanding
  template<range R>
    using const_iterator_t = const_iterator<iterator_t<R>>;                       // freestanding
  template<range R>
    using const_sentinel_t = const_sentinel<sentinel_t<R>>;                       // freestanding
  […]
}
```

3773. `views::zip_transform` still requires `F` to be `copy_constructible` when empty pack

Section: 26.7.24.1 [range.zip.transform.overview] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-09-12 **Last modified:** 2022-09-23

Priority: Not Prioritized

Discussion:

After P2494R2, range adaptors only require callable to be `move_constructible`, however, for `views::zip_transform`, when empty pack, it still requires callable to be `copy_constructible`. We should relax this requirement, too.

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4910.

1. Modify 26.7.24.1 [range.zip.transform.overview] as indicated:

> -2- The name `views::zip_transform` denotes a customization point object (16.3.3.3.6 [customization.point.object]). Let `F` be a subexpression, and let `Es...` be a pack of subexpressions.
>
> (2.1) — If `Es` is an empty pack, let `FD` be `decay_t<decltype((F))>`.
>
> (2.1.1) — If `move_constructible`~~`copy_constructible`~~`<FD> && regular_invocable<FD&>` is `false`, or if `decay_t<invoke_result_t<FD&>>` is not an object type, `views::zip_transform(F, Es...)` is ill-formed.
>
> (2.1.2) — Otherwise, the expression `views::zip_transform(F, Es...)` is expression-equivalent to
>
> ```
> ((void)F, auto(views::empty<decay_t<invoke_result_t<FD&>>>))
> ```
>
> (2.2) — Otherwise, the expression `views::zip_transform(F, Es...)` is expression-equivalent to `zip_transform_view(F, Es...)`.

3774. `<flat_set>` should include `<compare>`

Section: 24.6.5 [flat.set.syn] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2022-09-12 **Last modified:** 2022-09-23

Priority: Not Prioritized

Discussion:

This was originally editorial PR #5789 which is considered not-editorial.

`std::flat_set` and `std::flat_multiset` have `operator<=>` so `<compare>` should be included in the corresponding header, in the spirit of LWG 3330. `#include <compare>` is also missing in the adopted paper P1222R4.

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify the synopsis in 24.6.5 [flat.set.syn] as indicated:

```
#include <compare>              // see 17.12.1 [compare.syn]
#include <initializer_list>     // see 17.11.2 [initializer.list.syn]
```

3775. Broken dependencies in the *Cpp17Allocator* requirements

Section: 16.4.4.6.1 [allocator.requirements.general] **Status:** Tentatively Ready **Submitter:** Alisdair Meredith **Opened:** 2022-09-22 **Last modified:** 2022-10-12

Priority: Not Prioritized

View other active issues in [allocator.requirements.general].

View all other issues in [allocator.requirements.general].

Discussion:

This issue is extracted from P0177R2 as that paper stalled on the author's ability to update in time for C++17. While the issue was recorded and going to be resolved in the paper, we did not file an issue for the list when work on that paper stopped.

Many of the types and expressions in the *Cpp17Allocator* requirements are optional, and as such a default is provided that is exposed through `std::allocator_traits`. However, some types and operations are specified directly in terms of the allocator member, when really they should be specified allowing for reliance on the default, obtained through `std::allocator_traits`. For example, `X::pointer` is an optional type and not required to

exist; `XX::pointer` is either `X::pointer` when it is present, or the default formula otherwise, and so is guaranteed to always exist, and the intended interface for user code. Observe that bullet list in p2, which acts as the key to the names in the *Cpp17Allocator* requirements, gets this right, unlike most of the text that follows.

This change corresponds to the known implementations, which meet the intended contract rather than that currently specified. For example, `std::allocator` does not provide any of the `pointer` related typedef members, so many of the default semantics indicated today would be ill-formed if implementations were not already implementing the fix.

An alternative resolution might be to add wording around p1-3 to state that if a name lookup fails then the default formula is used. However, it is simply clearer to write the constraints as intended, in the form of code that users can write, rather than hide behind a layer of indirect semantics that may be interpreted as requiring another layer of SFINAE metaprogramming.

[2022-10-12; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 16.4.4.6.1 [allocator.requirements.general] as indicated:

`typename X::pointer`

-4- *Remarks*: Default: `T*`

`typename X::const_pointer`

-5- *Mandates*: `XX::pointer` is convertible to `XX::const_pointer`.

-6- *Remarks*: Default: `pointer_traits<XX::pointer>::rebind<const T>`

`typename X::void_pointer`
`typename Y::void_pointer`

-7- *Mandates*: `XX::pointer` is convertible to `XX::void_pointer`. `XX::void_pointer` and `YY::void_pointer` are the same type.

-8- *Remarks*: Default: `pointer_traits<XX::pointer>::rebind<void>`

`typename X::const_void_pointer`
`typename Y::const_void_pointer`

-9- *Mandates*: `XX::pointer`, `XX::const_pointer`, and `XX::void_pointer` are convertible to `XX::const_void_pointer`. `XX::const_void_pointer` and `YY::const_void_pointer` are the same type.

-10- *Remarks*: Default: `pointer_traits<XX::pointer>::rebind<const void>`

`typename X::value_type`

-11- *Result*: Identical to `T`.

`typename X::size_type`

-12- *Result*: An unsigned integer type that can represent the size of the largest object in the allocation model.

-13- *Remarks*: Default: `make_unsigned_t<XX::difference_type>`

`typename X::difference_type`

-14- *Result*: A signed integer type that can represent the difference between any two pointers in the allocation model.

-15- *Remarks*: Default: `pointer_traits<XX::pointer>::difference_type`

`typename X::template rebind<U>::other`

-16- *Result*: `Y`

-17- *Postconditions*: For all `U` (including `T`), `YY::~~template rebind~~rebind_alloc<T>~~::other~~` is `X`.

-18- *Remarks*: If `Allocator` is a class template instantiation of the form `SomeAllocator<T, Args>`, where `Args` is zero or more type arguments, and `Allocator` does not supply a `rebind` member template, the standard `allocator_traits` template uses `SomeAllocator<U, Args>` in place of `Allocator::rebind<U>::other` by default. For allocator types that are not template instantiations of the above form, no default is provided.

-19- [*Note 1*: The member class template `rebind` of `X` is effectively a typedef template. In general, if the name `Allocator` is bound to `SomeAllocator<T>`, then `Allocator::rebind<U>::other` is the same type as `SomeAllocator<U>`, where `SomeAllocator<T>::value_type` is `T` and `SomeAllocator<U>::value_type` is `U`. — *end note*]

[…]

`static_cast<X::pointer>(w)`

-29- *Result*: `X::pointer`

-30- *Postconditions*: `static_cast<X::pointer>(w) == p`.

`static_cast<X::const_pointer>(x)`

-31- *Result*: `X::const_pointer`

-32- *Postconditions*: `static_cast<X::const_pointer>(x) == q`.

`pointer_traits<X::pointer>::pointer_to(r)`

-33- *Result*: `X::pointer`

-34- *Postconditions*: Same as `p`.

`a.allocate(n)`

-35- *Result*: `X::pointer`

[…]

`a.allocate(n, y)`

-40- *Result*: `X::pointer`

[…]

`a.allocate_at_least(n)`

-43- *Result*: `allocation_result<X::pointer>`

-44- *Returns*: `allocation_result<X::pointer>{ptr, count}` where `ptr` is memory allocated for an array of count `T` and such an object is created but array elements are not constructed, such that `count == n`. If `n == 0`, the return value is unspecified.

[…]

[…]

`a.max_size()`

-50- *Result*: `X::size_type`

-51- *Returns*: The largest value `n` that can meaningfully be passed to ~~`X::`~~`a.allocate(n)`.

-52- *Remarks*: Default: `numeric_limits<size_type>::max() / sizeof(value_type)`

[…]

`a == b`

-59- *Result*: `bool`

-60- *Returns*: `a == Y::rebind_alloc<T>`~~`::other`~~`(b)`.

[…]

-92- An allocator type `X` shall meet the *Cpp17CopyConstructible* requirements (Table 33). The `X::pointer`, `X::const_pointer`, `X::void_pointer`, and `X::const_void_pointer` types shall meet the *Cpp17NullablePointer* requirements (Table 37). No constructor, comparison operator function, copy operation, move operation, or swap operation on these pointer types shall exit via an exception. `X::pointer` and `X::const_pointer` shall also meet the requirements for a *Cpp17RandomAccessIterator* (25.3.5.7) and the additional requirement that, when ~~`a`~~`p` and `(`~~`a`~~`p + n)` are dereferenceable pointer values for some integral value `n`, `addressof(*(`~~`a`~~`p + n)) == addressof(*`~~`a`~~`p) + n` is `true`.

-93- Let `x1` and `x2` denote objects of (possibly different) types `X::void_pointer`, `X::const_void_pointer`, `X::pointer`, or `X::const_pointer`. Then, `x1` and `x2` are equivalently-valued pointer values, if and only if both `x1` and `x2` can be explicitly converted to the two corresponding objects `px1` and `px2` of type `X::const_pointer`, using a sequence of `static_casts` using only these four types, and the expression `px1 == px2` evaluates to `true`.

-94- Let `w1` and `w2` denote objects of type `X::void_pointer`. Then for the expressions

```
w1 == w2
w1 != w2
```

either or both objects may be replaced by an equivalently-valued object of type `X::const_void_pointer` with no change in semantics.

-95- Let `p1` and `p2` denote objects of type `X::pointer`. Then for the expressions

```
p1 == p2
p1 != p2
p1 < p2
p1 <= p2
p1 >= p2
p1 > p2
p1 - p2
```

either or both objects may be replaced by an equivalently-valued object of type `X::const_pointer` with no change in semantics.

3778. `vector<bool>` missing exception specifications

Section: 24.3.12.1 [vector.bool.pspc] **Status:** Tentatively Ready **Submitter:** Alisdair Meredith **Opened:** 2022-09-13 **Last modified:** 2022-09-29

Priority: Not Prioritized

Discussion:

As noted back in P0177R2, the primary template for `vector` has picked up some exception specification, but the partial specialization for `vector<bool>` has not been so updated.

Several other changes have been made to `vector` in the intervening years, but these particular exception specifications have still not been updated. Note that the free-function `swap` in the header synopsis will automatically pick up this update once applied.

[2022-09-28; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 24.3.12.1 [vector.bool.pspc], partial class template `vector<bool, Allocator>` synopsis, as indicated:

```
namespace std {
  template<class Allocator>
  class vector<bool, Allocator> {
  public:
    […]
    // construct/copy/destroy
    constexpr vector() noexcept(noexcept(Allocator())) : vector(Allocator()) { }
    constexpr explicit vector(const Allocator&) noexcept;
    constexpr explicit vector(size_type n, const Allocator& = Allocator());
    constexpr vector(size_type n, const bool& value, const Allocator& = Allocator());
    template<class InputIterator>
      constexpr vector(InputIterator first, InputIterator last, const Allocator& = Allocator());
    template<container-compatible-range <bool> R>
      constexpr vector(from_range_t, R&& rg, const Allocator& = Allocator());
    constexpr vector(const vector& x);
    constexpr vector(vector&& x) noexcept;
    constexpr vector(const vector&, const type_identity_t<Allocator>&);
    constexpr vector(vector&&, const type_identity_t<Allocator>&);
    constexpr vector(initializer_list<bool>, const Allocator& = Allocator());
    constexpr ~vector();
    constexpr vector& operator=(const vector& x);
    constexpr vector& operator=(vector&& x)
      noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
               allocator_traits<Allocator>::is_always_equal::value);
    constexpr vector& operator=(initializer_list<bool>);
    template<class InputIterator>
      constexpr void assign(InputIterator first, InputIterator last);
    template<container-compatible-range <bool> R>
      constexpr void assign_range(R&& rg);
    constexpr void assign(size_type n, const bool& t);
    constexpr void assign(initializer_list<bool>);
    constexpr allocator_type get_allocator() const noexcept;

    […]
    constexpr iterator erase(const_iterator position);
    constexpr iterator erase(const_iterator first, const_iterator last);
    constexpr void swap(vector&)
      noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
               allocator_traits<Allocator>::is_always_equal::value);
    constexpr static void swap(reference x, reference y) noexcept;
    constexpr void flip() noexcept; // flips all bits
    constexpr void clear() noexcept;
  };
}
```

3781. The exposition-only alias templates *cont-key-type* and *cont-mapped-type* should be removed

Section: 24.6.1 [container.adaptors.general] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-09-16 **Last modified:** 2022-10-12

Priority: Not Prioritized

Discussion:

P0429R9 removed the `flat_map`'s single-range argument constructor and uses C++23-compatible `from_range_t` version constructor with alias templates *range-key-type* and *range-mapped-type* to simplify the deduction guide.

This makes *cont-key-type* and *cont-mapped-type* no longer useful, we should remove them.

[2022-10-12; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 24.6.1 [container.adaptors.general] as indicated:

> -7- The exposition-only alias template *iter-value-type* defined in 24.3.1 [sequences.general] and the exposition-only alias templates *iter-key-type* and *iter-mapped-type* defined in 24.4.1 [associative.general] may appear in deduction guides for container adaptors.
>
> ~~-8- The following exposition-only alias templates may appear in deduction guides for container adaptors:~~
>
> ```~~
> template<class Container>
>   using cont-key-type =                                  // exposition only
>     remove_const_t<typename Container::value_type::first_type>;
> template<class Container>
>   using cont-mapped-type =                               // exposition only
>     typename Container::value_type::second_type;~~
> ```

3782. Should `<math.h>` declare `::lerp`?

Section: 17.15.7 [support.c.headers.other] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2022-09-17 **Last modified:** 2022-10-12

Priority: Not Prioritized

View other active issues in [support.c.headers.other].

View all other issues in [support.c.headers.other].

Discussion:

According to 17.15.7 [support.c.headers.other]/1, `<math.h>` is required to provide `::lerp`, despite that it is a C++-only component. In P0811R3, neither `<math.h>` nor C compatibility is mentioned, so it seems unintended not to list `lerp` as an exception in 17.15.7 [support.c.headers.other].

Currently there is implementation divergence: libstdc++ provide `::lerp` in `<math.h>` but not in `<cmath>`, while MSVC STL and libc++ don't provide `::lerp` in `<math.h>` or `<cmath>` at all.

I'm not sure whether this should be considered together with LWG 3484. Since `nullptr_t` has become a part of the C standard library as of C23 (see WG14-N3042 and WG14-N3048), I believe we should keep providing `::nullptr_t` in `<stddef.h>`.

[2022-10-12; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 17.15.7 [support.c.headers.other] as indicated:

> -1- Every C header other than `<complex.h>` (17.15.2 [complex.h.syn]), `<iso646.h>` (17.15.3 [iso646.h.syn]), `<stdalign.h>` (17.15.4 [stdalign.h.syn]), `<stdatomic.h>` (33.5.12 [stdatomic.h.syn]), `<stdbool.h>` (17.15.5 [stdbool.h.syn]), and `<tgmath.h>` (17.15.6 [tgmath.h.syn]), each of which has a name of the form *<name>*`.h>`, behaves as if each name placed in the standard library namespace by the corresponding *<c name>* header is placed within the global namespace scope, except for the functions described in 28.7.6 [sf.cmath], the `std::lerp` function overloads (28.7.4 [c.math.lerp]), the declaration of `std::byte` (17.2.1 [cstddef.syn]), and the functions and function templates described in 17.2.5 [support.types.byteops]. […]

3784. `std.compat` should not provide `::byte` and its friends

Section: 16.4.2.4 [std.modules] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2022-09-19 **Last modified:** 2022-10-12

Priority: Not Prioritized

Discussion:

Currently 16.4.2.4 [std.modules]/3 seemly requires that the `std.compat` module has to provide `byte` (via `<cstddef>`), `beta` (via `<cmath>`) etc. in the global namespace, which is defective to me as these components are C++-only, and doing so would increase the risk of conflict.

I think we should only let `std.compat` provide the same set of global declarations as `<xxx.h>` headers.

[2022-10-12; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 16.4.2.4 [std.modules] as indicated:

> -3- The named module `std.compat` exports the same declarations as the named module `std`, and additionally exports declarations in the global namespace corresponding to the declarations in namespace `std` that are provided by the C++ headers for C library facilities (Table 26), except the explicitly excluded declarations described in 17.15.7 [support.c.headers.other].

3785. `ranges::to` is over-constrained on the destination type being a range

Section: 26.5.7.2 [range.utility.conv.to] **Status:** Tentatively Ready **Submitter:** Barry Revzin **Opened:** 2022-09-19 **Last modified:** 2022-09-28

Priority: Not Prioritized

View other active issues in [range.utility.conv.to].

View all other issues in [range.utility.conv.to].

Discussion:

The current wording in 26.5.7.2 [range.utility.conv.to] starts:

> If `convertible_to<range_reference_t<R>, range_value_t<C>>` is `true`

and then tries to do `C(r, args...)` and then `C(from_range, r, args...)`. The problem is that `C` might not be a range — indeed we explicitly removed that requirement from an earlier revision of the paper — which makes this check ill-formed. One example use-case is using `ranges::to` to convert a range of `expected<T, E>` into a `expected<vector<T>, E>` — `expected` isn't any kind of range, but it could support this operation, which is quite useful. Unfortunately, the wording explicitly rejects that. This change happened between R6 and R7 of the paper and seems to have unintentionally rejected this use-case.

[2022-09-28; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll. During telecon review we agreed that supporting non-ranges was an intended part of the original design, but was inadvertently broken when adding `range_value_t` for other reasons.

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.5.7.2 [range.utility.conv.to] as indicated:

> [*Drafting note:* We need to be careful that this short-circuits, since if `C` does not satisfy `input_range`, then `range_value_t<C>` will be ill-formed.]

```
template<class C, input_range R, class... Args> requires (!view<C>)
  constexpr C to(R&& r, Args&&... args);
```

-1- *Returns*: An object of type `C` constructed from the elements of `r` in the following manner:

(1.1) — If `C does not satisfy input_range or convertible_to<range_reference_t<R>, range_value_t<C>>` is `true`:

(1.1.1) — If `constructible_from<C, R, Args...>` is `true`:

```
C(std::forward<R>(r), std::forward<Args>(args)...)
```

(1.1.2) — Otherwise, if `constructible_from<C, from_range_t, R, Args...>` is `true`:

```
C(from_range, std::forward<R>(r), std::forward<Args>(args)...)
```

(1.1.3) — Otherwise, if

(1.1.3.1) — `common_range<R>` is `true`,

(1.1.3.2) — *`cpp17-input-iterator`*`<iterator_t<R>>` is `true`, and

(1.1.3.3) — `constructible_from<C, iterator_t<R>, sentinel_t<R>, Args...>` is `true`:

```
C(ranges::begin(r), ranges::end(r), std::forward<Args>(args)...)
```

(1.1.4) — Otherwise, if

(1.1.4.1) — `constructible_from<C, Args...>` is `true`, and

(1.1.4.2) — *`container-insertable`*`<C, range_reference_t<R>>` is `true`:

```
C c(std::forward<Args>(args)...);
if constexpr (sized_range<R> && reservable-container<C>)
  c.reserve(ranges::size(r));
ranges::copy(r, container-inserter<range_reference_t<R>>(c));
```

(1.2) — Otherwise, if `input_range<range_reference_t<R>>` is `true`:

```
to<C>(r | views::transform([](auto&& elem) {
  return to<range_value_t<C>>(std::forward<decltype(elem)>(elem));
}), std::forward<Args>(args)...);
```

(1.3) — Otherwise, the program is ill-formed.

3788. `jthread::operator=(jthread&&)` postconditions are unimplementable under self-assignment

Section: 33.4.4.2 [thread.jthread.cons] **Status:** Tentatively Ready **Submitter:** Nicole Mazzuca **Opened:** 2022-09-22 **Last modified:** 2022-10-12

Priority: 3

Discussion:

In the *Postconditions* element of `jthread& jthread::operator=(jthread&&)` (33.4.4.2 [thread.jthread.cons] p13), we have the following:

> *Postconditions*: `x.get_id() == id()`, and `get_id()` returns the value of `x.get_id()` prior to the assignment. `ssource` has the value of `x.ssource` prior to the assignment and `x.ssource.stop_possible()` is `false`.

Assume `j` is a joinable `jthread`. Then, `j = std::move(j);` results in the following post-conditions:

- Let `old_id = j.get_id()` (and since `j` is joinable, `old_id != id()`)
- Since `x.get_id() == id()`, `j.get_id() == id()`
- Since `get_id() == old_id`, `j.get_id() == old_id`
- Thus, `id() == j.get_id() == old_id`, and `old_id != id()`, which is a contradiction.

One can see that these postconditions are therefore unimplementable.

Two standard libraries – the MSVC STL and libstdc++ – currently implement `jthread`.

The MSVC STL chooses to follow the letter of the *Effects* element, which results in unfortunate behavior. It first `request_stop()`s, then `join()`s; then, it assigns the values over. This results in `j.get_id() == id()` – this means that `std::swap(j, j)` stops and joins `j`.

libstdc++ chooses instead to implement this move assignment operator via the move-swap idiom. This results in `j.get_id() == old_id`, and `std::swap(j, j)` is a no-op.

It is the opinion of the issue writer that libstdc++'s choice is the correct one, and should be taken into the standard.

[2022-09-28; Reflector poll]

Set priority to 3 after reflector poll.

[2022-09-28; Jonathan provides wording]

[2022-09-29; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 33.4.4.2 [thread.jthread.cons] as indicated:

```
jthread& operator=(jthread&& x) noexcept;
```

-12- *Effects*: If `&x == this` is `true`, there are no effects. Otherwise, if `joinable()` is `true`, calls `request_stop()` and then `join()`. ~~Assigns~~, then assigns the state of `x` to `*this` and sets `x` to a default constructed state.

-13- *Postconditions*: ~~`x.get_id() == id()` and~~ `get_id()` returns the value of `x.get_id()` prior to the assignment. `ssource` has the value of `x.ssource` prior to the assignment ~~and `x.ssource.stop_possible()` is `false`~~.

-14- *Returns*: `*this`.

3792. `__cpp_lib_constexpr_algorithms` should also be defined in `<utility>`

Section: 17.3.2 [version.syn] **Status:** Tentatively Ready **Submitter:** Marc Mutz **Opened:** 2022-10-05 **Last modified:** 2022-10-12

Priority: Not Prioritized

View other active issues in [version.syn].

View all other issues in [version.syn].

Discussion:

17.3.2 [version.syn] says that `__cpp_lib_constexpr_algorithms` is only defined in `<version>` and `<algorithm>`, but the macro applies to `std::exchange()` in `<utility>` as well (via P0202R3), so it should also be available from `<utility>`.

[2022-10-12; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 17.3.2 [version.syn] as indicated (It is suggested to retroactively apply to C++20):

```
[…]
#define __cpp_lib_concepts             202207L // also in <concepts>, <compare>
#define __cpp_lib_constexpr_algorithms 201806L // also in <algorithm>, <utility>
#define __cpp_lib_constexpr_bitset     202202L // also in <bitset>
[…]
```

3795. Self-move-assignment of `std::future` and `std::shared_future` have unimplementable postconditions

Section: 33.10.7 [futures.unique.future], 33.10.8 [futures.shared.future] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2022-10-19 **Last modified:** 2022-11-07

Priority: 3

View all other issues in [futures.unique.future].

Discussion:

The move assignment operators of `std::future` and `std::shared_future` have their postconditions specified as below:

> *Postconditions*:
>
> - `valid()` returns the same value as `rhs.valid()` returned prior to the assignment.
> - `rhs.valid() == false`.

It can be found that when `*this` and `rhs` is the same object and `this->valid()` is `true` before the assignment, the postconditions can't be achieved.

Mainstream implementations (libc++, libstdc++, msvc stl) currently implement such self-move-assignment as no-op, which doesn't meet the requirements in the *Effects:* element. As discussed in LWG 3788, I think we should say self-move-assignment has no effects for these types.

[2022-11-01; Reflector poll]

Set priority to 3 after reflector poll.

[2022-11-01; Jonathan provides wording]

[2022-11-07; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 33.10.7 [futures.unique.future] as indicated:

```
future& operator=(future&& rhs) noexcept;
```

-11- *Effects*: If `addressof(rhs) == this` is `true`, there are no effects. Otherwise:

(11.1) — Releases any shared state (33.10.5 [futures.state]).
(11.2) — move assigns the contents of `rhs` to `*this`.

-12- *Postconditions*:

(12.1) — `valid()` returns the same value as `rhs.valid()` prior to the assignment.
(12.2) — If `addressof(rhs) == this` is `false`, `rhs.valid() == false`.

2. Modify 33.10.8 [futures.shared.future] as indicated:

```
shared_future& operator=(shared_future&& rhs) noexcept;
```

-13- *Effects*: If `addressof(rhs) == this` is `true`, there are no effects. Otherwise:

(13.1) — Releases any shared state (33.10.5 [futures.state]).
(13.2) — move assigns the contents of `rhs` to `*this`.

-14- *Postconditions*:

(14.1) — `valid()` returns the same value as `rhs.valid()` returned prior to the assignment.
(14.2) — If `addressof(rhs) == this` is `false`, `rhs.valid() == false`.

```
shared_future& operator=(const shared_future& rhs) noexcept;
```

-15- *Effects*: If `addressof(rhs) == this` is `true`, there are no effects. Otherwise:

(15.1) — Releases any shared state (33.10.5 [futures.state]).
(15.2) — assigns the contents of `rhs` to `*this`.

[*Note 3*: As a result, `*this` refers to the same shared state as `rhs` (if any). — *end note*]

-16- *Postconditions*: `valid() == rhs.valid()`.

3796. *`movable-box`* as member should use default-initialization instead of copy-initialization

Section: 26.6.5.2 [range.repeat.view], 26.7.29.2 [range.chunk.by.view] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2022-10-20 **Last modified:** 2022-11-01

Priority: Not Prioritized

View other active issues in [range.repeat.view].

View all other issues in [range.repeat.view].

Discussion:

Currently, the member variable *`value_`* of `repeat_view` is initialized with the following expression:

```
movable-box<W> value_ = W();
```

which will result in the following unexpected error in libstdc++:

```
#include <ranges>

int main() {
  std::ranges::repeat_view<int> r; // error: could not convert '0' from 'int' to 'std::ranges::__detail::__box<int>'
}
```

This is because the single-argument constructors of the simple version of *`movable-box`* in libstdc++ are declared as `explicit`, which is different from the conditional `explicit` declared by the `optional`'s constructor, resulting in no visible conversion between those two types.

For MSVC-STL, the simple version of *`movable-box`* does not provide a single-argument constructor, so this form of initialization will also produce a hard error.

This may be a bug of existing implementations, but we don't need such "copy-initialization", the default-constructed *`movable-box`* already value-initializes the underlying type.

Simply using default initialization, as most other range adaptors do, guarantees consistency, which also eliminates extra move construction.

[2022-11-01; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.6.5.2 [range.repeat.view] as indicated:

```
namespace std::ranges {
  template<move_constructible W, semiregular Bound = unreachable_sentinel_t>
    requires (is_object_v<W> && same_as<W, remove_cv_t<W>> &&
              (is-integer-like<Bound> || same_as<Bound, unreachable_sentinel_t>))
   class repeat_view : public view_interface<repeat_view<W, Bound>> {
   private:
     // 26.6.5.3 [range.repeat.iterator], class repeat_view::iterator
     struct iterator;                              // exposition only

     movable-box<W> value_ = W();                  // exposition only, see 26.7.3 [range.move.wrap]
     Bound bound_ = Bound();                       // exposition only
     […]
   };
   […]
}
```

2. Modify 26.7.29.2 [range.chunk.by.view] as indicated:

```
namespace std::ranges {
  template<forward_range V, indirect_binary_predicate<iterator_t<V>, iterator_t<V>> Pred>
    requires view<V> && is_object_v<Pred>
  class chunk_by_view : public view_interface<chunk_by_view<V, Pred>> {
    V base_ = V();                                          // exposition only
    movable-box<Pred> pred_ = Pred();                       // exposition only

    // 26.7.29.3 [range.chunk.by.iter], class chunk_by_view::iterator
    class iterator;                                         // exposition only
    […]
  };
  […]
}
```

3798. Rvalue reference and `iterator_category`

Section: 25.3.2.3 [iterator.traits] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2022-10-22 **Last modified:** 2022-11-01

Priority: Not Prioritized

View other active issues in [iterator.traits].

View all other issues in [iterator.traits].

Discussion:

Since C++11, a forward iterator may have an rvalue reference as its `reference` type. I think this is an intentional change made by N3066. However, several components added (or changed) in C++20/23 recognize an iterator as a *Cpp17ForwardIterator* (by using `forward_iterator_tag` or a stronger iterator category tag type as the `iterator_category` type) only if the `reference` type would be an lvalue reference type, which seems a regression.

[2022-11-01; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 25.3.2.3 [iterator.traits] as indicated:

-2- The definitions in this subclause make use of the following exposition-only concepts:

```
[…]
template<class I>
concept cpp17-forward-iterator =
  cpp17-input-iterator<I> && constructible_from<I> &&
  is_lvalue_reference_v<iter_reference_t<I>> &&
  same_as<remove_cvref_t<iter_reference_t<I>>,
          typename indirectly_readable_traits<I>::value_type> &&
  requires(I i) {
    {  i++ } -> convertible_to<const I&>;
```

```
    { *i++ } -> same_as<iter_reference_t<I>>;
  };
[…]
```

2. Modify 26.7.9.3 [range.transform.iterator] as indicated:

> -2- The member *typedef-name* `iterator_category` is defined if and only if *Base* models `forward_range`. In that case, *`iterator`*`::iterator_category` is defined as follows: Let `C` denote the type `iterator_traits<iterator_t<`*`Base`*`>>::iterator_category`.
>
> (2.1) — If `is_~~lvalue_~~reference_v<invoke_result_t<`*`maybe-const`*`<Const, F>&, range_reference_t<`*`Base`*`>>>` is `true`, then
>
> (2.1.1) — if `C` models `derived_from<contiguous_iterator_tag>`, `iterator_category` denotes `random_access_iterator_tag`;
>
> (2.1.2) — otherwise, `iterator_category` denotes `C`.
>
> (2.2) — Otherwise, `iterator_category` denotes `input_iterator_tag`.

3. Modify 26.7.15.3 [range.join.with.iterator] as indicated:

> -2- The member *typedef-name* `iterator_category` is defined if and only if *`ref-is-glvalue`* is `true`, and *`Base`* and *`InnerBase`* each model `forward_range`. In that case, *`iterator`*`::iterator_category` is defined as follows:
>
> (2.1) — […]
>
> (2.2) — If
>
> `is_~~lvalue_~~reference_v<common_reference_t<iter_reference_t<`*`InnerIter`*`>, iter_reference_t<`*`PatternIter`*`>>>`
>
> is `false`, `iterator_category` denotes `input_iterator_tag`.
>
> (2.3) — […]

4. Modify 26.7.24.3 [range.zip.transform.iterator] as indicated:

> -1- The member *typedef-name* *`iterator`*`::iterator_category` is defined if and only if *`Base`* models `forward_range`. In that case, *`iterator`*`::iterator_category` is defined as follows:
>
> (1.1) — If
>
> `invoke_result_t<`*`maybe-const`*`<Const, F>&, range_reference_t<`*`maybe-const`*`<Const, Views>>...>`
>
> is not a~~n lvalue~~ reference, `iterator_category` denotes `input_iterator_tag`.
>
> (1.2) — […]

5. Modify 26.7.26.3 [range.adjacent.transform.iterator] as indicated:

> -1- The member *typedef-name* *`iterator`*`::iterator_category` is defined as follows:
>
> (1.1) — If `invoke_result_t<`*`maybe-const`*`<Const, F>&, `*`REPEAT`*`(range_reference_t<`*`Base`*`>, N)...>` is not a~~n lvalue~~ reference, `iterator_category` denotes `input_iterator_tag`.
>
> (1.2) — […]

3801. `cartesian_product_view::`*`iterator`*`::`*`distance-from`* ignores the size of last underlying range

Section: 26.7.31.3 [ranges.cartesian.iterator] **Status:** Tentatively Ready **Submitter:** Patrick Palka **Opened:** 2022-10-25 **Last modified:** 2022-11-04

Priority: Not Prioritized

View other active issues in [ranges.cartesian.iterator].

View all other issues in [ranges.cartesian.iterator].

Discussion:

The helper *`scaled-size`*`(N)` from the wording for P2374R4's `cartesian_product_view::`*`iterator`*`::`*`distance-from`* is recursively specified as:

> Let *`scaled-size`*`(N)` be the product of `static_cast<difference_type>(ranges::size(std::get<`*`N`*`>(`*`parent_->bases_`*`)))` and *`scaled-size`*`(N + 1)` if *`N`* `< sizeof...(Vs)`, otherwise `static_cast<difference_type>(1)`;

Intuitively, *`scaled-size`*`(N)` is the size of the cartesian product of all but the first *`N`* underlying ranges. Thus *`scaled-size`*`(sizeof...(Vs))` ought to just yield the size of the last underlying range (since there are `1 + sizeof...(Vs)` underlying ranges), but according to this definition it yields `1`. Similarly at

the other extreme, *scaled-size*(0) should yield the product of the sizes of all the underlying ranges, but it instead yields that of all but the last underlying range.

For cartesian_product_views of two or more underlying ranges, this causes the relevant operator- overloads to compute wrong distances, e.g.

```
int x[] = {1, 2, 3};
auto v = views::cartesian_product(x, x);
auto i = v.begin() + 5;  // *i == {2, 3}
assert(*i == tuple{2, 3});
assert(i - v.begin() == 5); // fails, expects 3, because:
// scaled-sum = scaled-distance(0) + scaled-distance(1)
//            = ((1 - 0) * scaled-size(1)) + ((2 - 0) * scaled-size(2))
//            = 1 + 2 instead of 3 + 2
```

The recursive condition should probably use <= instead of <.

[2022-11-04; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4917.

1. Modify 26.7.31.3 [ranges.cartesian.iterator] as indicated:

```
template<class Tuple>
  constexpr difference_type distance-from(Tuple t);
```

-7- Let:

(7.1) — *scaled-size*(*N*) be the product of static_cast<difference_type>(ranges::size(std::get<*N*>(*parent_->bases_*))) and *scaled-size*(*N* + 1) if *N* ≤< sizeof...(Vs), otherwise static_cast<difference_type>(1);

(7.2) — *scaled-distance*(*N*) be the product of static_cast<difference_type>(std::get<*N*>(*current_*) - std::get<*N*>(t)) and *scaled-size*(*N* + 1); and

(7.3) — *scaled-sum* be the sum of *scaled-distance*(*N*) for every integer 0 ≤ N ≤ sizeof...(Vs).